

Data-efficient Reinforcement Learning for Non-stationary Fed-batch Processes

Rokas Pranevičius



Master of Science
School of Informatics
University of Edinburgh
2026

Abstract

Optimising industrial bioprocess controls of cell cultivation remains an ongoing challenge, since manual recipe-driven strategies require extensive specialist knowledge. Reinforcement learning (RL) offers a flexible alternative, but is typically sample-inefficient, requiring unrealistically many agent–environment interactions for convergence, which limits its practical application. We adapt MC-PILCO, a data-efficient Gaussian Process (GP) model-based policy-search algorithm, to a non-stationary, data-limited fed-batch penicillin fermentation process. We test whether splitting two known biological phases, growth and production, across two GP models captures the non-stationarity of cellular dynamics within-episode better than a single GP.

Our results indicate that MC-PILCO transfers successfully, matching a Bayesian-optimisation (BO) baseline in yield while achieving it more reliably, without constraint violations, and more data-efficiently. Additionally, when explicit time was added to the GP as an additional state—acting as a proxy for phase dynamics as indicated by our delayed feed experiments—it outperforms BO further. Our dual-GP model, however, does not improve industrial bioprocess controls over a single GP. Tests reveal that this is likely due to sugar feed control having limited influence on the simulator-defined biological variables. To our knowledge, this is the first PILCO-family application to fed-batch fermentation processes, and we suggest that future work on enriching the action space is a more promising direction.

Table of Contents

1	Introduction	1
1.1	Research Questions	2
2	Literature Review	4
2.1	Fed-batch processes	4
2.2	Advanced Process Controls (APC)	5
2.3	Reinforcement Learning	6
2.4	PILCO-based methods	7
2.5	MC-PILCO	10
2.6	Non-stationary dynamical systems	11
2.7	Biological Characteristics and Multi-Phase Modeling	13
3	Methodology	15
3.1	Reinforcement Learning Environment	15
3.1.1	Environment Setup	15
3.1.2	Action	15
3.1.3	States	17
3.1.4	Penalties and Reward Function	19
3.2	Algorithm	21
3.2.1	Initial Exploration	21
3.2.2	Policy	21
3.2.3	Uncertainty propagation	21
3.2.4	Implementation and Hyperparameters	22
3.3	Growth and Production phases	22
3.4	Gradient Flow	24
3.5	Experimental design	25
3.5.1	Evaluation protocol and metrics	25

3.5.2	Baseline	26
3.5.3	Feed-delay disturbance experiment	26
4	Results and Evaluation	28
4.1	Single-GP performance (RQ1)	28
4.2	Phase decomposition (RQ2)	32
4.2.1	Dual-GP Decomposition Results	32
4.2.2	Relaxing the data and prior assumptions	33
4.2.3	Assessing Process Failure: Single-Step Feed Effects and Batch Variation	34
4.2.4	Biological Interpretation	35
4.2.5	Remarks on single and dual-GP decomposition	37
5	Conclusion and Discussion	38
5.1	Generalisability	39
5.2	Limitations and Future Work	40
	Bibliography	41
A	Supplementary Results	48
A.1	Substrate operating point on the production curve	54

Chapter 1

Introduction

Cell cultivation represents a challenging industrial process for many applications. Particularly, penicillin fermentation involves stochastic biological cellular dynamics, which imposes difficulties for optimising the bioreactor controls of its cell cultures. Penicillin remains an important antibiotic produced at industrial scale, while similar fed-batch processes are used in the manufacture of products including insulin, monoclonal antibodies, and vaccines [14, 50, 6]. Originally, batch processes for fermentation were closed systems, adding nutrients at the initial stage with no further modifications. However, current industrial fermentation processes are often operated as fed-batch processes, where nutrients are incrementally added to the bioreactor, creating an open system with closed-loop feedback-based controls [28].

Fed-batch processes inspired the development of Advanced Process Controls (APC), which has enabled the continuous monitoring and adjusting of process variables in real time to maintain optimal conditions within the bioreactor. APC's application has been shown to achieve higher penicillin production or increased reliability [31]. Industrial batches can run for hundreds of hours in large-volume bioreactors [18], meaning that a single inaccurate control can lead to batch collapse, while even modest yield improvements can accumulate across an entire production campaign. Batch processes are time- and resource-expensive, driving the need for more data-efficient, robust, and, importantly, reliable solutions.

Recent APC techniques for refinement of penicillin fermentation include the introduction of machine learning to the process controls. However, there are limitations to its application: bioreactors inherently allow for only partial observability of their contents, and the biological processes themselves exhibit nonlinear behaviour and non-stationarity across and within batches, which presents challenges for stationary methods [21].

One particularly attractive APC framework is Reinforcement Learning (RL), which is highly adaptable to non-stationary dynamics. Simethy et al. [47] claims that a carefully chosen model-free RL method can outperform conventional controllers in penicillin production to achieve higher yields. However, model-free RL methods, where agents must learn through continuous direct interactions, impose a fundamental challenge on real-world bioreactors: the policy learning phase requires expansive exploration, which increases the risk of irreversible process failures through the violation of physical or biological constraints. Additionally, model-free algorithms can be inefficient, as evident in Zhang et al. [57] benchmark report, where they fail to outperform a sample-efficient Bayesian Optimisation (BO) baseline on the fed-batch penicillin task.

An alternate method addressing these shortcomings is Model-based Reinforcement Learning (MBRL) [30], which constructs an explicit model of the system dynamics to be utilised for policy optimisation, reducing the number of exploration steps required. The main challenge of MBRL is choosing the right controls to model the underlying biological complexities to obtain a sufficiently accurate model of the system dynamics.

1.1 Research Questions

In this dissertation, we explore the application of the model-based RL policy-search algorithm MC-PILCO [3] to optimise penicillin fermentation. MC-PILCO is attractive for this problem because it allows for a more data-efficient solution as opposed to model-free and other model-based approaches. Additionally, its Gaussian Processes that provide inherent predictive uncertainty, allowing for safer policy optimization data-efficient approach where data is extremely sparse. Despite MC-PILCO demonstrating advantages in robotics, it was primarily designed for stationary systems. To the best of our knowledge, no prior study explores whether PILCO-family methods can be adapted to fed-batch processes, where dynamics are inherently non-stationary.

Research Question 1: Can MC-PILCO, a model-based RL policy-search method, be adapted to improve fed-batch processes to reliably achieve a higher yield without violating the underlying biological cellular dynamics, causing batch collapse, and operating under severe data constraints.

In short, we utilise MC-PILCO to model the system dynamics within the bioreactor using Gaussian Processes (GPs) through imaginary particle-based rollouts within the

RL framework to optimise the next action within the fermentation process. Actions are judged against this real-time inferred GP model based on the defined rewards. We then compare it against a Bayesian Optimisation baseline [57].

Model-based approaches, including MC-PILCO, accommodate many controls for RL actions. We chose substrate feed rate F_s (sugar) because it is the standard manipulated variable for yield maximisation in fed-batch penicillin control [24]. However, modelling substrate feed rate as the main control requires careful consideration of physical and biological constraints such as vessel capacity and broth viscosity – ensuring the overall penicillin yield is obtained reliably under data and safety constraints.

Another big challenge of applying model-based methods to industrial fermentation processes is modelling the various distinct biological phases. While microorganisms may exhibit different dynamical processes, we focus on two main distinct biological phases for penicillin fermentation, namely a growth phase in which biomass accumulates, and a production phase in which the fungus *P. chrysogenum* synthesises penicillin [5]. These two phases exhibit different dynamics, creating intra-episode non-stationarity.

Potential solutions to accommodate the non-stationarity between these two phases using a single GP are, firstly, to include time as an observable state to carry the phase temporal information, or secondly, to learn the two-phase distinct dynamics through a non-stationary kernel. While these are valid choices, stationary GP averages different phases dynamics, disregarding their differences. Given that some microorganisms, including *P. chrysogenum*, exhibit dynamics that change within a single batch, this begs the question of how GPs can best model phase dynamics that exhibit this inter-phase/intra-episode non-stationarity.

Research Question 2: Can injecting biological phase information into MC-PILCO’s RL framework as two phase-specific GP dynamics models address this intra-episode non-stationarity better than a single GP, and what limitations may constrain this approach?

We explore this biologically motivated alternate model, exploiting MC-PILCO’s Monte Carlo Sampling to inject this known phase structure directly, allowing us to represent within-episode non-stationarity with two independently trained GP dynamics models which correspond to the growth and production phases respectively. We then compare it to a single GP (with and without time as a GP input).

Chapter 2

Literature Review

2.1 Fed-batch processes

Bioreactors are usually run in three operational modes: Batch, where all necessary medium components are added at the beginning of fermentation, Continuous, where all necessary nutrients are being fed continuously, and **Fed-Batch** where the nutrients are fed either intermittently or continuously via feeding streams during the fermentation process. The Fed-batch processes showed superior performance for certain cell productions, such as *Penicillium chrysogenum* fungus, which produces Penicillin [28]. Fed-batch processes are particularly well positioned for us, as it allows for feedback-based closed-loop controls (§ 2.3).

The fermentation process in real-world bioreactors requires time and expensive resources, and as a result, fed-batch simulators aim to close this gap. IndPenSim [18] is one such publicly available fed-batch simulator that is currently used as a benchmark in many process systems analysis and control studies. It simulates 100,000 L bioreactor used for an industrial strain of *Penicillium chrysogenum*, and was built on an existing [36] structured penicillin fermentation model. It was later extended to Goldrick et al. [19] with the addition of a Raman spectroscopy device, allowing for the observation of some delayed (*offline*) variables in a real-time (*online*) manner, enabling more sophisticated APC solutions. Goldrick et al. [19] also extends to historical batches, including their collapses and failure modes, which indicate safety bounds when controlling the vessel, and shows batch variability. Notably, we will also use a Raman spectroscopy extension by assuming observations of delayed states online (§ 3.1.3), allowing us to eliminate delayed confounders, although data from previous batches could not be re-used here, as explained in § 3.1.1.

Simulating bioreactor systems poses significant challenges. Living cells are inherently stochastic, with minor environmental differences such as nutrient gradients causing behaviour to deviate, resulting in nonlinear, high-dimensional, and time-varying dynamics. Some states are unobserved or observed with delay. These factors contribute to a gap between simulated and real cells, limiting their cross-comparability. Furthermore, simulators differ in solvers, step control, and other design choices. Thus, instead of a policy assumed to transfer between simulators or to real organisms, generalisability must come from the method itself rather than any single simulator.

2.2 Advanced Process Controls (APC)

Historically, fed-batch bioprocesses have relied on Proportional-Integral (PI) and Derivative (PID) feedback control, while Advanced Process Control (APC) methods such as MPC, adaptive control, fuzzy control, and soft-sensor-based control have been introduced to improve prediction, optimisation, adaptation, and handling of unmeasured process variables [4, 26, 29, 32]. In particular, MPC can predict future process trajectories and explicitly incorporate process constraints [23]. However, these approaches fundamentally depend on either an accurate process model, predefined control knowledge, or sufficient representative data for model development and tuning. Therefore, their performance can degrade when the process model or prior knowledge does not adequately represent the underlying process.

Model-free Reinforcement Learning (RL) provides an alternative by learning optimal control (policy) directly from process interactions, without requiring an explicit process model. This allows the controller to learn nonlinear control strategies directly from observed process responses and avoids the need to specify a complete mechanistic model. Simethy et al. [47] show that this flexibility comes at the cost of data efficiency, as model-free RL relies on exploration to discover effective controls, resulting in many trial-and-error interactions. Within bioreactors, where processes are time- and resource-expensive, exploration is particularly challenging because biological processes can be difficult or impossible to reverse once an inappropriate control is applied [33].

Model-based RL (MBRL) addresses this limitation by exploiting an explicit model of the process to predict the consequences of potential actions. By combining model-based prediction with policy learning, it can reduce direct process interaction while retaining the ability to adapt within this fixed control model. This makes MBRL a promising framework for data-efficient control of expensive fed-batch processes.

Furthermore, Zhang et al. [57] has shown that naive offline and model-free RL methods fail to outperform Bayesian Optimisation (BO), which uses a surrogate to model the dynamics for the PenSimEnv task, which indeed suggests that MBRL may be better suitable for fed-batch processes. Therefore, in this dissertation, we specifically focus on model-based RL methods and use BO as our baseline, which we will try to improve upon in terms of yield and reliability, measured by collapsed batches across different seeds and obtained yield variance.

2.3 Reinforcement Learning

Reinforcement learning is a type of machine learning where an agent learns to make decisions by interacting with the environment. Unlike supervised learning, where models are trained from labelled examples, RL does not require demonstrations of the correct action. Instead, the agent repeatedly interacts with its environment through the iterative trial-and-error process and observes the consequences of its actions based on the reward received. This RL ability to optimise long-term *cumulative future reward* makes RL particularly suitable for complex sequential control problems. It adjusts (learns) its behaviour by optimising the *policy*, which guides the agent on the optimal actions to take at any given time. We can denote deterministic policy as $A_t = \pi(S_t)$, where π stands for policy. This interaction-based learning makes RL particularly attractive for sequential optimisation problems, where decisions made at one time influence future system behaviour. As a result, RL has been successfully applied across domains such as games, robotics, autonomous systems and industrial process control, where deriving optimal decision-making strategies analytically is often infeasible [45]. The interaction between the agent and its environment is typically modelled as a sequential decision-making problem. More formally, assume the agent is in state S_t (which in our case will be a vector of current simulator variables at time t) and takes an action A_t . The agent thus influences the environment to change by taking an action, resulting in a new S_{t+1} for which it gets rewarded R 2.1.

One often-used formalisation for RL is the Markov Decision Process (MDP), which follows from the Markov property 2.1,

$$P(X_{t+n}|X_t, X_{t-1}, \dots) = P(X_{t+n}|X_t) \quad (2.1)$$

saying that the future state only depends on the current state. This assumption enables the agent to make decisions based solely on the current state, allowing policies to be

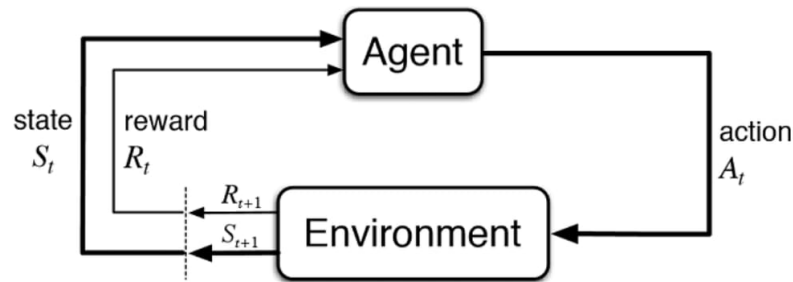


Figure 2.1: Agent-environment interaction in a Markov decision process (Fig. 3.1 [48])

mathematically solvable and computationally efficient. This assumption in fed-batch processes is often violated due to partial observability, whose effect we will try to minimise (§ 3.1.3).

RL algorithms are commonly categorised according to whether they utilise an explicit model of the environment. Model-free methods learn policies or value functions directly from experience without estimating the underlying system dynamics, whereas model-based methods either assume access to, or learn, a model of the environment to support planning and policy improvement. Model-free approaches are generally simpler and more widely used, but they obtain rewards by iterative interactions with the environment. Simethy et al. [47] showed that although the optimisation process takes seconds per episode, these constant agent-environment interactions demand lots of training iterations to converge. Additionally, model-free methods are infeasible to deploy for real bioreactors since any poor actions risk costly resources. Model-free RL is limited by low data efficiency as its naive exploration extends batch times, risks biological constraint violations, potentially ruining the entire batch due to biological irreversibility [33]. Other studies addressed these limitations by injecting required domain knowledge via recipe warm-start initialisation, resulting in faster policy convergence and a reduced number of episodes needed [27]. Thus, ongoing research aims to find a reliable control solution demanding a low number of episodes before convergence.

2.4 PILCO-based methods

Obtaining a usable dynamics model is the main challenge for applying MBRL to fed-batch bioprocesses. As biological systems are nonlinear and stochastic, with organism-specific kinetics and metabolic transitions, the dynamics must be learned from interaction under severe data limitations, where each experiment must remain safe

in vivo while still being informative.

Deisenroth and Rasmussen [11] address this setting by learning a probabilistic Gaussian Process (GP) dynamics model and propagating its predictive uncertainty through the state trajectory during policy evaluation. Rather than testing candidate policies on the real system, PILCO evaluates them on the learned model itself, computing the expected cost and gradient from its predictions, so that a policy is executed only *after* it has been optimised. A common failure mode of MBRL under limited data is model bias, where the model is trusted beyond what the data supports. PILCO reduces this by accounting for model uncertainty, and optimising against an uncertainty-aware model rather than actually exploring. This makes it a viable candidate for a fed-batch setting where naive exploration risks irreversible batch loss.

Both PILCO and MC-PILCO (§ 2.5) consider a discrete-time system governed by a single *unknown* transition function f . Following the formulation of Amadio et al. [3],

$$x_{t+1} = f(x_t, u_t) + w_t, \quad w_t \sim \mathcal{N}(0, \Sigma_w), \quad (2.2)$$

where $x_t \in \mathbf{R}^{d_x}$ is the state, $u_t \in \mathbf{R}^{d_u}$ the action, and w_t zero-mean Gaussian *process* noise with $\Sigma_w = \text{diag}(\sigma_{w,1}^2, \dots, \sigma_{w,d_x}^2)$ (one variance per state channel). The objective is to optimise the parameters θ of a parameterised policy π_θ so that the expected cumulative cost $J(\theta) = \sum_{t=0}^T \mathbf{E}_{x_t} [c(x_t)]$ over horizon T is minimised, with $x_0 \sim p(x_0)$.

Initial transitions are collected under exploratory controls, and an independent GP per state dimension is fitted to the one-step dynamics. Given a state-action input $q_t = [x_t, u_t]$, the GP predicts a distribution over the state change rather than an absolute value (Eq. 2.3).

$$p(\Delta_t | q_t) = \mathcal{N}(\mu_\Delta(q_t), \Sigma_\Delta(q_t)), \quad \Delta_t = x_{t+1} - x_t. \quad (2.3)$$

Modelling the dynamics as state changes captures the system dynamics directly, and the predicted variance $\Sigma_\Delta(q_t)$ makes model uncertainty explicit, growing in unexplored regions of the state space (Eq. 2.4).

The GP uses a zero prior mean and a squared-exponential (SE) kernel as its covariance function, and learns its hyperparameters through marginal likelihood maximisation. In regions where no training data is available, the GP defaults its one-step mean prediction to $x_{t+1} = x_t$. We will keep this behaviour unchanged, as it contributes to a more stable model.

The predictive variance follows from the GP posterior. For a single query input q_t ,

the one-step posterior variance is

$$\Sigma_{\Delta}^a(q_t) = \underbrace{k_a(q_t, q_t)}_{\text{prior variance}} - \underbrace{k_a(q_t, \tilde{Q}) [K_a(\tilde{Q}, \tilde{Q}) + \sigma_{\epsilon, a}^2 I]^{-1} k_a(\tilde{Q}, q_t)}_{\text{reduction from data}}, \quad (2.4)$$

where $\tilde{Q}$ are the training inputs and $a = 1, \dots, d_x$ indexes the independent GP fitted to each state channel. This is the variance of the GP's estimate of the dynamics function f itself. It is the epistemic part of the uncertainty, meaning it comes from limited data and shrinks as more trajectories are collected. The $\sigma_{\epsilon}^2 I$ term inside the inverse enters as the likelihood noise of the regression. It allows the GP to fit noisy training data smoothly rather than interpolating every point exactly, and it is learned jointly with the kernel hyperparameters by marginal likelihood maximisation. The subtracted term depends on $k(q_t, \tilde{Q})$, the SE-kernel similarity between q_t and the training data. The SE kernel is defined as

$$k(q_t, q') = \sigma_f^2 \exp\left(-\frac{1}{2l^2} \|q_t - q'\|^2\right), \quad (2.5)$$

and measures how similar two inputs q_t and q' are, depending only on the distance $\|q_t - q'\|$ between them (as closer inputs give higher covariance). Two hyperparameters set the shape of this similarity: the length-scale l is the distance over which inputs stay correlated, and thus sets how far the training data reduces uncertainty and how smooth the function is assumed to be, while the signal variance σ_f^2 is the value of the kernel when $q_t = q'$, and thus sets the uncertainty. Because this similarity decays with distance, near the training data $k(q_t, \tilde{Q})$ is large, and the variance is driven toward zero, whereas far from it $k(q_t, \tilde{Q}) \rightarrow 0$ and the variance returns to the prior σ_f^2 . The predictive variance therefore grows as q_t moves away from the training data.

Equation (2.4) is the variance of the latent function only. The predictive variance of a new, noisy transition adds the observation noise once more, on top of the whole expression,

$$\tilde{\Sigma}_{\Delta}(q_t) = \Sigma_{\Delta}(q_t) + \sigma_w^2, \quad (2.6)$$

where this outer σ_w^2 is the noise on the state transition itself (Eq. (2.2)), not the shape of the kernel. The distinction matters whenever the propagated uncertainty is meant to describe realisable trajectories rather than the model's belief about f (we describe its practical consequences in § 3.2.3).

The action is $u_t = \pi_{\theta}(x_t)$, a deterministic policy parameterised as a linear or radial basis function (RBF) neural network with learnable parameters θ . PILCO optimises θ directly by differentiating the expected cumulative cost through the dynamics model, without a separately learned value estimate.

During policy evaluation, long-term recursive one-step prediction models $p(x_1), \dots, p(x_T)$ are obtained by feeding each predicted state distribution back as an uncertain input, so the one-step variance of Eq. (2.4) compounds across the horizon. Propagating a Gaussian input $p(q_t)$ through the GP gives

$$p(\Delta_t) = \int p(f(q_t) | q_t) p(q_t) dq_t, \quad (2.7)$$

which *critically* is non-Gaussian and analytically intractable. PILCO approximates $p(\Delta_t)$ using *moment matching*, which yields a Gaussian distribution that is propagated to the next step. The approximation is closed-form and differentiable, so the policy gradient $\nabla_{\theta} J$ is computed analytically. However, this introduces two restrictions [3]: (i) the single-Gaussian approximation represents only a unimodal state distribution and cannot capture multiple outcomes; and (ii) the moments are tractable only for the SE kernel and differentiable cost functions.

Our work is concerned with intra-episode non-stationarity, where biomass accumulates, nutrients deplete, and metabolism shifts between growth and production phases as the governing dynamics change within a single episode. This violates PILCO’s assumption of a single, time-invariant f (Eq. (2.2)), which averages both regimes and as we hypothesise, may degrade predictive accuracy. Addressing it requires relaxing the restrictions above; for instance, a phase-dependent, blended kernel [7], which in PILCO setup is constrained by the SE-only moment matching. Therefore, we will further see how the particle-based MC-PILCO method (§ 2.5) removes these restrictions, admitting any differentiable dynamics model inside the rollout (a structured, non-stationary kernel, or, as we use in our work – a blend of two phase-specific GPs).

2.5 MC-PILCO

Existing PILCO-based extensions each relax some of PILCO’s restrictions, but none explicitly targets the intra-episode non-stationarity that is central to this work. Chatzilygeroudis et al. [8] use a gradient-free optimisation method, which relaxes the differentiable-cost requirement and permits non-differentiable cost functions, but it does not make structured kernels or phase blends available to the dynamics model. Gal et al. [16] develop Deep-PILCO, replacing the GP with a Bayesian neural network to capture discontinuous, non-smooth dynamics, addressing functional expressiveness. However, it refits a Gaussian to the particles at each step, reintroducing the unimodal approximation, and has been reported to require more trials than PILCO to learn a task, reducing the

data efficiency.

Amadio et al. [3] instead relaxes the restrictions imposed by moment matching. Rather than approximating the intractable integral of Eq. (2.7) with a closed-form Gaussian, MC-PILCO evaluates it with *particle-based rollouts*, propagating a set of M sampled particles through the one-step model. Because the state distribution is now represented by particles rather than a single Gaussian, the SE-kernel and unimodal-state restrictions no longer apply, and any differentiable kernel or cost function could be used. This also allows arbitrary differentiable dynamics models inside the rollout, allowing us to impose dual-phase structure later (§ 2.7). The policy gradient $\nabla_{\theta}J$ is then estimated by Monte Carlo. The *reparameterisation trick* makes each sampled state a differentiable function of θ , so the expected cost is approximated by averaging over particles and differentiated by backpropagation.

A similar particle-based, reparameterised approach was explored earlier in Parmas et al. [35], but was found infeasible because the estimated gradient exploded in magnitude. MC-PILCO overcomes this through two design choices, cost-function shaping (§ 3.1.4) and dropout on the policy during optimisation, which together stabilise the gradient. Furthermore, on the cart-pole benchmark, MC-PILCO reduces the cumulative cost faster than PILCO, which makes it attractive for the low-data fed-batch regime.

The trade-off of this flexibility is computational cost. Sampling replaces the closed-form variance with a per-particle evaluation of the GP posterior variance (Eq. (2.4)), whose data-dependent term scales as $O(n^2)$ in the number of training points n . The total cost is $O(d_x MT n^2)$, where d_x is the state dimension, M the number of particles, and T the control horizon. The terms M , T , and n therefore require careful control and considerate state selection (§ 3.1.3).

Note that non-stationarity is not solved by the MC-PILCO itself. It only removes the SE-only restriction and thereby makes non-stationary and phase-blended kernels available to be injected into the dynamics model. To the best of our knowledge, we could not find prior work on addressing within-episode non-stationarity with MC-PILCO; thus, we aim to understand if we can use this flexibility in fed-batch controls.

2.6 Non-stationary dynamical systems

Broadly, we distinguish non-stationarity by *when* the dynamics change. *Intra-episode* non-stationarity means the dynamics vary within a single episode, whereas *inter-episode* means they change across episodes. Within an episode, the change may be gradual

or abrupt. For across-episode change, Iten et al. [20] propose an adaptive data-buffer mechanism that limits the influence of old data through forgetting and sliding-window techniques to maintain calibrated uncertainty, while others infer the relationship between tasks through a hierarchical latent-variable model [43]. In fed-batch bioprocesses, non-stationarity appears both across and within episodes, but the scope of this dissertation is the intra-episode case. Inter-episode variation is not modelled and is instead absorbed into the GP’s learned noise, which we return to in § 4.2, where it emerges as the main limitation.

Literature on intra-episode non-stationarity is dense, and most of it learns the change online. FANS-RL [15] learns time-evolving latent change factors and a factored causal graph, which requires extensive agent-environment interaction. Change-point methods such as MBCD [2] detect the transition online and switch context models, incurring a detection delay. The MOLe approach of Nagabandi et al. [34] discovers task structure online, but requires a meta-training task distribution and abundant interaction. In our project, we neither learn, detect, nor discover the change within the episode. We anticipate it offline and inject a biological prior on the phase structure into a data-scarce GP controller.

Intra-episode non-stationarity can be spurious, where the dynamics only appear to change because the state is incomplete, through an unmeasured variable, poor state representation, or hidden history, which makes the dynamics non-Markovian, or *intrinsic*, where the underlying dynamics change through an exogenous time-varying driver or physical drift in the plant parameters. In fed-batch processes, the two are hard to separate, because the latent variables that drive the change are observable only offline, with a delay, or not at all. A change that is genuinely state-triggered, and therefore Markovian in the full state, is indistinguishable from intrinsic drift if the triggering variable is unobserved. We show in § 3.3 that IndPenSim’s within-batch regime changes are genuinely state-triggered.

The common remedy is to restore the Markov property by augmenting the GP input with a variable that carries the missing information. One choice is time itself, which recovers part of the Markovian structure but is fragile if the regime timing varies from batch to batch. A more robust choice is an informative, non-temporal variable, such as a lagged state or an explicit phase indicator, which exposes the hidden regime directly. Alternatively, the change can be absorbed into the GP model, either through a non-zero mean function or semi-parametric kernel, so that the GP learns only the residual around a physical prior, or through a non-stationary kernel whose length-scale varies to allow

the smoothness of the function to change over the trajectory. A further option, and the one we pursue, is to leave each model stationary and instead decompose the episode, fitting a separate GP per phase and blending them across the transition.

What distinguishes our approach is that the prior information is injected from the known biology of the system rather than learned online, which is what preserves data efficiency. Our central assumption is that the biology of the system, morphological or physiological, is sufficient to locate the phase transition offline, so the phase structure can be supplied to the controller in advance rather than inferred from data.

2.7 Biological Characteristics and Multi-Phase Modeling

Many microorganisms are grown via fed-batch processes, including *E. coli* [52], *S. cerevisiae* [9], and *P. chrysogenum*, exhibit distinct phase-dependent kinetic behaviour. However, the nature of the growth-to-production transition varies across organisms. For instance, in *Pichia pastoris*, the shift from growth to recombinant-protein production is triggered by operator switching the carbon feed from glycerol to methanol [58], so the phase boundary is known by construction. In penicillin fermentation, differentiation of growing into non-growing regions is driven by the mean culture age [18], so the transition emerges from the culture’s own history. So the transition must be inferred rather than set by the operator’s actions.

Although the specific dynamics, transition conditions, and phase durations differ between organisms, fed-batch processes commonly exhibit growth and secondary-metabolite production phases. Structured models of *P. chrysogenum* resolve the biomass further, into growing (A_0), non-growing (A_1), degenerated (A_3) and autolysed (A_4) regions [18], with Paul and Thomas [36] additionally describing vacuolation. These are morphological compartments that coexist within a batch rather than sequential phases, but their relative dominance shifts over the batch. This gives the growth/production split its mechanistic basis. In this work, we restrict the process to two phases, *growth* and *production*, motivated by their broad biological distinction across fed-batch processes, and by the data cost of considerations, as each additional split halves the transitions available to fit each GP, which is prohibitive in our low-data regime.

Kluge et al. [25] reports a significant shift from nitrogen to carbon limitation around the transition between growth and production, alongside phase-dependent changes in

the dominant uptake. Similarly, Ryu and Hospodka [42] suggests that in real cultures, during the idiophase, the specific growth rate must be maintained at approximately ($> 0.015 \text{ h}^{-1}$) to sustain production capacity while avoiding excessive biomass accumulation. These observations indicate that the relationship between process variables and their rates of change is not constant throughout the batch, providing a biological basis for considering the fermentation dynamics as phase-dependent. Early simulators such as Bajpai and Reuss [5] represented biomass growth and penicillin formation using separate kinetic expressions. The dynamic relationship between growth and penicillin production is also known to change throughout fermentation, with relationships observed under steady-state conditions not necessarily remaining valid under dynamic fed-batch operation [12]. In contrast, most RL formulations do not explicitly encode this biological phase structure in the dynamics model or control policy. Phase-aware approaches have been proposed, such as the phase-segmentation strategy of Yoo et al. [55], but these primarily use phase information to structure learning across batches. Therefore, we investigate whether explicitly incorporating within-episode phase transitions can improve learning and control efficiency.

Jiang et al. [21] provide further evidence that splitting the Penicillin fermentation process into distinct phases can improve its process modelling. This motivates the hypothesis that a similar decomposition may also benefit control optimisation. More generally, phase splitting provides a way of representing the globally evolving process using locally specialised models. Rasmussen and Ghahramani [38] show that mixtures of Gaussian process experts can represent complex functions using multiple GPs, each specialising to different regions of the input space. Although their approach uses a learned gating mechanism rather than predefined biological phases, it provides a conceptual basis for representing globally non-stationary behaviour through locally specialised GP models. However, a learned gate must be estimated from data online, which is the data cost we avoid by fixing the gate from biology before training (§ 3.3).

In this work, biological phase information is therefore used as an offline structural prior to partition the within-episode dynamics into two locally defined regimes. Rather than assuming that the dynamics are stationary across the entire batch, separate GP models are used to represent the dynamics within the growth and production phases. The resulting hypothesis is that this decomposition can reduce the modelling burden associated with representing the globally non-stationary process, while retaining the data-efficient learning properties of the GP-based controller.

Chapter 3

Methodology

3.1 Reinforcement Learning Environment

3.1.1 Environment Setup

Originally, the IndPenSim simulator was developed in MATLAB ([18, 19]), but for flexibility and third-party library compatibility, we use the Python port, PenSimPy (developed by Zhang [56]), integrating with SciPy’s LSODA solver. However, the two implementations diverge numerically. Simethy [46] in Table 3 report a yield misalignment of 1421 kg against 1808 kg over a 120 h batch attributable to solver choice alone. Batch lengths also differ, up to ≈ 317 h in the MATLAB case studies against a fixed 230 h in PenSimPy, so absolute yields are not comparable across implementations.

A further inconsistency concerns how recipe setpoints are read at each timestep. PenSimPy holds the previous value until the next breakpoint, whereas MATLAB applies the upcoming breakpoint’s value across the entire preceding interval (Appendix A.1). This affects every recipe-driven control (F_s , F_{oil} , F_g , F_{water} , F_{PAA} , $F_{pressure}$), but is particularly damaging during discharge, which operates in a few large harvests at fixed batch times. We instead follow the MATLAB behaviour, as it is verified by multiple studies, by changing `left_sp` to `right_sp` when reading the recipe.

We therefore cannot reuse the historical IndPenSim batches and have to regenerate all recipe baselines under our own simulator version.

3.1.2 Action

F_s is the primary manipulated nutritional input, affecting penicillin through different direct and indirect pathways. Rumi et al. [41] discretised the action space, which

allowed only finite candidate actions. Kim et al. [24] instead kept the feed continuous, using differential dynamic programming to optimise the full feed trajectory on a nominal model.

Furthermore, instead of allowing policy to fluctuate within an undefined range, we follow Johannink et al. [22], which enforces the action controls on the residual of the prior knowledge of the system at time t , which is our nominal fed-batch recipe, resulting in more sample-efficient controls [27]. We allow the policy to fluctuate between $\pm 50\%$ within the recipe, which allows the agent to be flexible and not deviate too much from the conservative recipe. Critically, Yang and Das [53] have demonstrated that residual can augment a nominal controller even when the controlled process is subject to time-varying dynamics. Thus, we define our action as

$$u_t = \underbrace{u_t^{\text{recipe}}}_{\text{recipe prior}} \overbrace{(1 + \alpha a_t)}^{\text{RL correction}} \quad (3.1)$$

where $\alpha = 0.5$ is a fixed constant, and $a_t \in [-1, 1]$ is agent selected action ($a_t = 0$ means following recipe). We used multiplicative residual as it scales corrections appropriately and guarantees non-negative feed, and 50% of the residual was used to give the action enough flexibility, while maintaining safe controls. Thus, the agent learns how and when to deviate from the existing recipe, which provides a strong prior over feasible feed trajectories, rather than exploring the full range of unrestricted absolute feed actions, making the policy safer and converging quicker, accounting for a data-efficient regime.

Within IndPenSim’s own 12-minute simulation time step, Simethy et al. [47] found that a 2h action step outperformed a 1h step. Based on this result, we considered increasing the action step to 5h to reduce the number of experiments required. Our preliminary experiments showed that 2h steps allowed more flexible control, but provided only a marginally higher yield and a lower one-step MSE (Appendix A.3, A.5). Given that MC-PILCO scales as $O(d_x M T n^2)$ in horizon T , we chose 5h action steps. We therefore were able to use a larger number of training and evaluation runs during our experimentation, rather than marginal improvements in yield or prediction error (resulting in ≈ 46 steps per batch).

Time remains implicitly present in the action space through the time-varying recipe in the residual-control parameterisation. Thus, in § 4, when we refer to ‘*excluding time*’, we mean only its exclusion as an explicit GP input. This does not limit the extent to which we evaluate multiphase experiments, since we are interested in whether adding

explicit phase structure to the dynamics representation provides additional benefit beyond the temporal information already available through the action. Thus, we retain the recipe-based residual-action formulation as it ensures safety and data efficiency, which are paramount in a low-data fermentation setting. Agents exploring without recipe reference resulted in approximately a third of batches violating the constraints (Tab. A.8).

3.1.3 States

Although the IndPenSim simulator allows for a flexible state selection, irrelevant and highly correlated states add noise to the model, increase the computational GP cost, and complicate deployability. Thus, we aim for a minimal sufficient set, which would keep the state space approximately Markovian Eq. (2.1), and allow us to express the reward.

In real-world fed-batch processes, states are online (measured directly), offline (delayed), or unobservable. Initially, from Table 1 [19] we selected states directly controlled by our single F_s action, that is, Weight (Wt), Penicillin concentration (P), Biomass (X) and substrate (S). Weight is directly measurable, while P and X measures are delayed. Since S cannot be measured, we excluded it as it is not real-world deployable. Although Goldrick et al. [19] show Raman spectroscopy application to measure phenylacetic acid (PAA) online with effectively no delay, Golabgir and Herwig [17] further show that real-time measurements of penicillin concentration (P), can be acquired every few seconds to minutes following appropriate chemometric calibration such as partial least squares (PLS) regression. Authors further indicate that combining Raman spectroscopy with respiratory rate measurements and a particle filter-based soft sensor enables online estimation of biomass concentration (X). Thus, we make an assumption that both P and X can be estimated online using Raman spectroscopy combined with process modelling and state estimation techniques, and add it to our state space.

From our transition function definition in Eq. 2.2, if the state x_t is incomplete, the same input may lead to a different outcome at different points in the batch, resulting in variation that cannot be learned from the observed state. Since the policy $\pi(s)$ acts on the current state alone, two batches sharing an observed state receive the same action even when their underlying conditions differ. To make the state more representative of the Markov condition, we therefore additionally include viscosity (*Visc*).

Viscosity plays a central role in fermentation because it affects how efficiently oxy-

gen is transferred into the broth, which in turn determines the dissolved oxygen available to the cells. Viscosity is also indirectly affected by substrate feeding F_s , as substrate promotes biomass growth, which can increase broth viscosity and subsequently affect oxygen transfer. By including viscosity in the state, we capture the feedback explicitly, allowing the effect of substrate feeding on penicillin production to be explained through observed state variables rather than being treated as unexplained variation.

On a real bioreactor, viscosity measurements are usually delayed. Direct real-time sensors, such as the vibrating-rod instrument of Picque and Corrieu [37], allow viscosity to be estimated from signals the reactor already records, such as agitator power or stirrer torque [13]. These methods are empirical and usually need to be calibrated for the specific process, but they show that a real-time viscosity signal is physically obtainable in stirred-tank reactors generally. Therefore, we include viscosity and assume it as an online metric.

Therefore, we finalise our minimal sufficient set of state space that is approximately Markovian as biomass (X), penicillin concentration (P), weight (W_t), and viscosity (V). In addition, Time is used as another GP input state for *some* of the models to capture non-stationary dynamics of the system.

For each state, we consider linear or logarithmic encoding, defined by an encoding function g , and examine the resulting per-step difference $\Delta z = g(x_{t+1}) - g(x_t)$. For a GP to train well, these differences should keep a roughly constant scale throughout the batch, so that a single learned noise term neither over- nor under-represents the variation of the inputs. To choose an encoding, the state variables were analysed over multiple simulated recipe-based batch trajectories, comparing how consistently the encoded per-step changes behaved across different stages of the batch.

The analysis showed that a logarithmic encoding is more suitable for vessel weight (W_t), biomass (X), and penicillin concentration (P). These states grow multiplicatively over their operating range, so under a linear encoding, the step sizes are spread out, whereas the logarithmic encoding converts this constant relative growth into steps of roughly constant size. Viscosity, by contrast, was retained on a linear scale. Although it also varied over a wide range, the control-relevant region is when the viscosity is high, risking batch collapse. A logarithmic encoding would compress this upper region, reducing the encoding's sensitivity, and would place the channel on the exponential decode that is prone to numerical instability during rollouts. We then normalised each channel to $[-1, 1]$ interval, which helped unify the ranges of each state, as well as make lengthscale priors appropriate for each state.

Operating bounds are taken from Table 1 of Goldrick et al. [19] and are used as cost penalties (§ 3.1.4).

3.1.4 Penalties and Reward Function

The penalty values were selected by considering the penalty-to-reward ratio, which balances task completion against constraint violations. This can be expressed as $(\hat{r}(s, a) = r(s, a) - \frac{1}{\lambda} c(s, a))$, where λ controls the relative weight of constraint penalties. $\lambda \gg 1$ indicates an overly conservative agent, favouring staying close to the real recipe, while with $\lambda \ll 1$, the agent prioritises yield at the risk of constraint violations, resulting in less safe exploration. We normalised the penalties to allow a single constraint strength parameter λ to control their overall magnitude.

The penalties account for vessel overflow, excessive viscosity, abrupt changes in feed rate, and outcome spread across particles. Soft penalties were used for the physical constraints to provide a gradual penalty before reaching the hard bounds, allowing the policy to account for approaching constraint violations rather than only penalising violations after they occur. The penalties were scaled relative to the penicillin mass at risk, placing the constraint costs in the same units as the process objective.

Different constraint strengths were tested on a penicillin concentration-dense cost function (Tab. A.5). Although a strength of 0.75 achieved the highest mean yield, it operated closer to the viscosity and vessel capacity limits. λ of 1.5 resulted in 2/5 under-performing batches. Given that the objective is reliable control, we selected a more conservative λ of 0.5. This also achieved the same highest penicillin concentration of 28.2 g/L while maintaining a greater distance from the constraint boundaries and achieving a higher yield-to-weight density.

For reward design, Amadio et al. [3] show that a highly selective cost can produce regions with very small or zero gradients when the predicted state remains far from the target, causing optimisation to get stuck in local minima. Conversely, a less selective cost provides gradients over a wider region but may give a weaker incentive to reach the target. Therefore, the cost should provide sufficient gradient information without being overly selective or tailored to a specific operating regime, which could restrict policy generalisation.

The reward function must also be checked for reward hacking, where optimisation of a proxy objective leads to behaviour that does not maximise the true objective. To assess this, we utilise Bayesian Optimisation to perform a proxy optimisation robustness

test on generated batches.

Using paired seeds, BO was used to optimise two objectives separately. The first maximised the true objective, total penicillin yield, giving the best-found baseline $a_A^* = \operatorname{argmax}_a Y(a)$, while the second maximised the reward function under test, $a_B^* = \operatorname{argmax}_a R(a)$. The resulting proxy gap, therefore, is $\Delta Y = Y(a_A^*) - Y(a_B^*)$, and reward functions producing a smaller ΔY were considered more aligned with the true objective. Paired statistical tests were then used to assess whether differences in performance were statistically significant. During optimisation, actions were applied in a closed loop to better reflect the behaviour of the final policy.

Studies on penicillin show that a dense concentration function outperforms a terminal penicillin mass function in maximising the yield for some model-free RL algorithms (Table 1. [47]). On a different model-free approach, Li et al. [27] demonstrates successful application of a penicillin dense mass increment function by incorporating in-batch harvest.

In addition to the latter, we further hypothesise that rewarding the growth rate (ΔP) might better shape the policy behaviour than rewarding pure concentration reward, as it provides denser learning signals. We also tried rewarding terminal vessel mass as a direct proxy for final yield. We thus test the following reward function designs:

- **Penicillin Concentration dense:** $p_weight * P$
- **Penicillin Concentration change:** $p_weight * (P_t - P_{t-1})$
- **Penicillin mass terminal:** $p_weight * P_T * W_T / 1000$
- **Penicillin mass change:** $p_weight * \Delta(P_t V_t) / 1000 + \text{discharge}$.

After performing the reward tests (Fig. A.6) we obtain only two valid cost functions: Penicillin Concentration dense reward and Penicillin mass change cost. Although we tested our models on both, in § 4 we will be mainly using the **Penicillin Concentration dense** cost function. From our investigations, we found that with the concentration change cost, the agent was overfeeding the vessel at the start, causing batch collapse due to increased viscosity in centipoise (cP). We also found that the Penicillin mass terminal reward was unsuccessful, as it does not account for discharge and provides only a terminal learning signal, making credit assignment more difficult.

3.2 Algorithm

3.2.1 Initial Exploration

We build an initial GP state model based on the initial explorations. In this project, we make an assumption that either five historical batches are available, or the agent can perform five initial exploration episodes itself. Furthermore, instead of Li et al. [27] requiring safe representative batches, our GP models can have failed ones, as it allows the model to cover more data points, which better shapes uncertainty regions. However, instead of historical batches, we decided to use explorations. To more closely replicate real action fluctuations and introduce more variance, we change exploration control every 20h in an open-loop manner, and scale it at $\pm 50\%$ of recipe action residual.

3.2.2 Policy

PILCO optimises a deterministic policy by propagating through the GP dynamics via moment matching, which restricts it to closed-form kernels and a unimodal state distribution [11]. MC-PILCO [3] instead propagates particles through the GP and optimises the policy by reparameterised gradients through the sampled rollout, removing the closed-form restriction, which allows for non-stationary kernels or a blend of separately fitted GPs as the dynamics model. We use this particle-rollout optimisation to minimise the expected cumulative cost $J(\theta)$ over the parameters θ of a squashed radial-basis-function policy π_θ , with $u_{\max} = 1$.

$$a_t = \pi_\theta(x_t) = u_{\max} \tanh\left(\frac{1}{u_{\max}} \sum_{i=1}^{100} w_i \exp\left(-\left\|\frac{x_t - c_i}{\ell}\right\|^2\right)\right), \quad (3.2)$$

Therefore the policy output is the bounded $a_t \in [-1, 1]$, and is applied by the agent as the residual correction of Eq. 3.1 (§ 3.1.2). Its learnable parameters are the basis-function weights w_i , centers c_i , and the shared lengthscale vector ℓ , i.e. $\theta = \{w_i, c_i, \ell\}$.

3.2.3 Uncertainty propagation

Eq. (2.6) proved important in our implementation. The variance propagated to the particles omitted this outer noise term, carrying only the epistemic term of Eq. (2.4), omitting the outer noise term of Eq. (2.6) (§2.2 Williams and Rasmussen [51]). Since the training data concentrates along the operating trajectory, this epistemic term collapses toward zero in rollout revisited states, resulting in no spread within sampled particles.

In our multi-step evaluations, the resulting bands were overconfident, with 10–90% band coverage of only 0.09–0.37 (Fig. A.1). We modified this by sampling particles from the noisy predictive distribution, using each channel’s learned likelihood noise $\sigma_{\epsilon,a}^2$ as the estimate of the process noise $\sigma_{w,a}^2$ in Eq. (2.6), requiring no manual tuning, which restored band coverage to 0.70–1.00 (Fig. A.2) across all our measured states.

3.2.4 Implementation and Hyperparameters

We optimise the policy with $M = 400$ particles over the full horizon. After every step the particles are clamped to $[-1, 1]$, which keeps them within the range the GPs were trained on. We also add a small amount of noise (normalised $\sigma = 0.01$) to the training targets of every channel except time, whose increment Δ is always fixed. Training uses Adam, starting with a dropout rate of $p_d = 0.25$ and halving the learning rate whenever the cost stops improving, until it reaches a floor and the trial ends. Table A.2 lists the remaining settings.

3.3 Growth and Production phases

To inject phase information, we make two assumptions. First, that the microorganism genuinely exhibits distinct phase behaviour within a single trajectory. Second, that the transition between phases can be inferred from available signals. Indicators of the transition can be *morphological*, based on the physical structure of the biomass (A_0 , A_1 , K_{diff}), or *physiological*, based on what the culture is doing rather than what it is made of (CER, OUR, substrate S).

We first inspect the process equations of IndPenSim [19] to verify that the simulator exhibits state-triggered within-batch non-stationarity. Of the 33 state derivatives, only viscosity carries an explicit time term, and even that produces a gradual drift rather than a regime switch (its time-dependent factor rises monotonically from 0.25 to 0.28 over the batch). The genuine regime changes are instead gated by state thresholds, including the differentiation constant K_{diff} , reaction heat Q_{rxn} , pH, and viscosity itself, which controls the oxygen transfer coefficient. This confirms dynamics change within a batch, and that the change tracks biological state, motivating our biological-based phase transition, rather than splitting only based on time.

We first consider a **morphological** split. Goldrick et al. [18] describe the growth-to-production transition as the differentiation of growing regions A_0 into non-growing,

production-capable regions A_1 , at a rate

$$r_{\text{diff}} = \frac{\mu_{\text{diff}} A_0}{K_{\text{diff}} + s}, \quad (3.3)$$

where the differentiation constant K_{diff} declines with mean cell age A_t until it saturates,

$$K_{\text{diff}}(t) = \max(0.75 - 0.006A_t(t), 0.09), \quad A_t(t) = \text{CultureAge}(t)/X(t). \quad (3.4)$$

From Eq. 3.3, a smaller K_{diff} increase the differentiation rate, thus 0.09 indicates *maximal* age-driven differentiation, reached at $A_t \approx 110\text{h}$. Therefore, transition is bounded by the peak of A_0 , where net growth ends and differentiation begins to dominate, and the K_{diff} floor where the age-driven differentiation rate saturates. We note that operating substrate stays far below K_{diff} throughout the batch, so its inhibition in Eq. 3.3 is negligible, indicating that the transition is governed by cell age.

According to our measurements, two markers differed substantially. A_0 peak near 65 h and the K_{diff} floored near 149, (Table A.8). To reflect this, we blend the two phase-specific GPs across the $[A_0, K_{\text{diff}}]$ interval using the sigmoid weight of Eq. 3.6, consistent with Jiang et al. [21], who blend adjacent-stage models, but instead using inverse distance weighting. We indicate A_0 peak to be less reliable of the two markers, as it can present several local peaks and is shaped by many factors at once (falling K_{diff} , substrate depletion, A_1 feedback, and dilution). Thus, we slightly increase our blend width to reflect these variations.

Although morphological split allows for split accurate transition detection, both A_0 and K_{diff} are *internal* ODE states, unobservable on a real plant. The morphological transition is therefore computed offline from statistics of over 20 recipe-driven batches (Table A.8), and applied as a fixed split, identical across every batch.

We therefore also consider a split from information available at run time. Among the tested indicators, the carbon evolution rate CER tracked the production phase best, identifying it more consistently than biomass, time, or oxygen uptake. However, CER is not part of the observed state, and is computed from the gas-phase CO_2 balance (Eq. 32, [18]) rather than from the state directly. Since respiratory CO_2 output is driven by the active biomass in the vessel, CER tracks biomass mass closely, correlating with the observed product $X \cdot W_t$ at $\rho = 0.991$ across recipe-driven batches (agreeing with §2.8 Goldrick et al. [18] saying that $\text{CER} \propto \text{biomass-mass}$). We therefore approximate it by an observable biomass-mass pivot

$$\text{BM}(t) = \frac{X(t) W_t(t)}{1000}. \quad (3.5)$$

We split each batch at the first decision step, where the running maximum of $\text{BM}(t)$ crosses a threshold. The running maximum is needed because raw BM peaks near 134 h and then declines, so a plain threshold would re-cross late in the batch. Because X and W_t are directly observed (§ 3.1.3), BM can also be evaluated during rollouts, making this an adaptive per-batch split rather than a fixed one.

To summarise, the morphological pivot (Type 1) is computed offline and fixed at the sigmoid centre of the $[A_0, K_{\text{diff}}]$ interval, ≈ 100 h. The biomass-weight pivot is dynamic, applied either per training episode (Type 2) or additionally at every rollout step, where each particle’s trajectory is individually split (Type 3).

We acknowledge that a more faithful split would allow each state variable its own transition time and sharpness, as they evolve on different timescales [21]. However, locating several boundaries reliably is difficult under observed batch variability (Tab. A.8), complicating the interpretability. Thus, we keep it out of scope in this project.

3.4 Gradient Flow

The dual-phase dynamics model represents the transition between the two biological phases as a smooth combination of two independently trained GP transition functions, $f_1(\cdot)$ and $f_2(\cdot)$ (2.2). Rather than switching fixed at the phase boundary, we interpolate their predictions with a phase-dependent weight,

$$f = (1 - w) f_1 + w f_2, \quad w = \sigma(k(\xi - \xi_{\text{piv}})), \quad (3.6)$$

where $w \in [0, 1]$ is a logistic sigmoid centered on the phase pivot ($w \approx 0.5$), ξ is the phase coordinate where k controls the smoothness of the transition.

The predictive variances combine under the same weights, as the mixture of the two phase distributions,

$$\Sigma = (1 - w) \Sigma_1 + w \Sigma_2 + w(1 - w) (\mu_1 - \mu_2)^2, \quad (3.7)$$

where μ_i and Σ_i are predicted by f_i , and the final term is the between-phase spread. Treating the two as independent samples would instead give $(1 - w)^2 \Sigma_1 + w^2 \Sigma_2$, halving the propagated variance at $w = 0.5$, which we measured as a source of under-coverage across the transition. Each Σ_i already carries its process noise (§ 3.2.3). Blending is applied in the encoded representation of § 3.1.3, so for the log-scaled channels (W_t , X , P) it is a weighted geometric mean of the two phases’ predictions in physical units.

Before the pivot ($w \approx 0$) the first GP dominates, and after pivot ($w \approx 1$) the second GP weights more. This construction is important for MC-PILCO, because it obtains its policy gradient by backpropagating the objective through the sampled particle rollouts. Replacing the single transition of Eq. (2.2), the blended rollout step becomes

$$x_{t+1} = (1 - w_t) f_1(x_t, u_t) + w_t f_2(x_t, u_t). \quad (3.8)$$

When w_t is fixed, the gradient through transition is a weighted combination of both GPs,

$$\frac{\partial x_{t+1}}{\partial u_t} = (1 - w_t) \frac{\partial f_1}{\partial u_t} + w_t \frac{\partial f_2}{\partial u_t}, \quad (3.9)$$

so the policy is optimised using information from both phases while the phase transition time is left unchanged.

For all our phase-splits, the pivot ξ_{piv} is a fixed hyperparameter and is never updated by the optimiser. Morphological and physiological split configurations, therefore, only differ in the phase coordinate ξ . In a fixed time split ($\xi_t = t T_{\text{step}}$), the weight depends only on the decision index T_{step} , so w_t 's gradients w.r.t. the policy is zero and Eq. (3.9) applies directly. With a biomass-based (dynamical) pivot, ξ_t is decoded from each particle's state, so w_t becomes state-dependent and would add an extra gradient term,

$$\frac{\partial x_{t+1}}{\partial w_t} = f_2(x_t, u_t) - f_1(x_t, u_t), \quad (3.10)$$

This would allow the policy to influence *which GP is trusted* by steering the particle state, rather than choosing actions that genuinely improve the trajectory. We therefore detach this weight from the computation before evaluating Eq. (3.8).

The weight still sets the per-particle phase mixture, but this dependence on the state no longer contributes to the gradient. The policy can still shift *when* a rollout crosses the pivot. By feeding more or less aggressively, it changes the biomass trajectory and hence the crossing time. However, this occurs only as an indirect consequence of controlling biomass, which the cost already governs, and never through a direct gradient on the routing itself.

3.5 Experimental design

3.5.1 Evaluation protocol and metrics

Baseline single-phase and multi-phase models (§ 4) are trained on three seeds and evaluated on ten held-out seeds, giving 30 seed $\times$ batch evaluations per configuration.

Seeds are shared across all policies to allow cross-model comparison. Our training budget is five exploration episodes with ten policy-search trials, and the recipe baseline is policy-independent, evaluated on the same ten seeds. Although some models were tested on both valid cost functions (Penicillin Dense and Mass Change), the main analysed models are trained on Penicillin Dense.

The primary metric is total penicillin yield (kg), reported as either absolute or Δ against the recipe of the same seed, since absolute yield shifts under disturbance (Eq. 3.12). Effect sizes are expressed relative to inter-batch variation, taken from 100 recipe-driven batches (Table A.1). Two types of violations are *counted* rather than averaged into the total yield: peak viscosity above 100 cP and vessel weight above 1.1×10^4 kg, as identified by Goldrick et al. [19].

Model quality is assessed separately from control performance. One-step prediction is reported as RMSE normalised by target standard deviation, with R^2 . Predictive uncertainty is assessed by the ratio of GP band half-width to the observed 10–90% replicate spread (calibrated = 1). All baselines and models are regenerated under the same solver and recipe handling (§ 3.1.1), so comparisons are valid without our own project, but not directly comparable to published IndPenSim results.

3.5.2 Baseline

Bayesian optimisation (BO) was selected as a sample-efficient black-box optimisation baseline due to its suitability for expensive and data-limited optimisation problems. It provides a strong benchmark for assessing whether the proposed GP-based MBRL approach can exploit limited process data more effectively than a method that directly optimises the batch-level objective without explicitly modelling the underlying process dynamics. To match our setup, we set F_s control to $\pm 50\%$ of the recipe residual, while the rest of the actions are recipe and PID-controlled. Under otherwise identical settings, we instead allow F_s control to be changed every 25 h rather than once at the start (as per Zhang et al. [57]). This, for us, resulted in the *highest yield* achieving baseline.

3.5.3 Feed-delay disturbance experiment

In a real fermentation, the growth-to-production phase transition does not occur at a fixed hour. It varies with inoculum and batch conditions. Studies identify time as one potential solution to this non-stationarity; however, we need to test if policy learned actual dynamics rather than a proxy to it. Therefore, we design an experiment to see the

extent to which learned policies that receive explicit time as a GP input, generalise to new environmental conditions, in particular, simulated delayed phase transition.

We displace the transition by time-shifting the nutritional component of the recipe. At each simulation step the F_s and F_{oil} setpoints are read Δ hours earlier than the current time, while every other manipulated variable (aeration, pressure, discharge, and water) continues to follow the original schedule (with all remaining inputs held at $\text{recipe}(t)$):

$$F_s(t) = \text{recipe}_{F_s}(t - \Delta), \quad F_{oil}(t) = \text{recipe}_{F_{oil}}(t - \Delta), \quad (3.11)$$

We only shift F_s and F_{oil} as they are nutritional inputs responsible for growth so the culture receives its original feed recipe in the correct sequence but later, without starving the culture. Because the recipe holds its first setpoint for any earlier time, each batch starts at the initial feed rate of 8 L/h and stays there for Δ hours before the shifted recipe takes effect. Phenylacetic acid (PAA) and F_{water} were left on the original schedule, as PAA is held at a fixed setpoint by a PID loop that overrides any shift, and water acts on the constrained vessel weight rather than on nutrition.

Keeping the non-nutritional inputs on their original timing delays the culture while the rest of the operation continues as scheduled, as would happen on a plant where a slow batch is not rescheduled, and discharge with aeration still follows original recipe timings. Time then becomes misleading, since it no longer matches the state of the culture. Therefore, this experiment reveals whether a policy has learned the transition itself or only when it usually occurs.

We delay the feed by $\Delta \in \{0, 20, 40, 60, 80\}$ h, which was verified to move the measured biomass pivot from 63 h to 132 h (Table A.7), confirming that it desynchronises the true transition rather than perturbing feed magnitude. The disturbance itself is applied by the environment rather than through the action channel, so every policy controls the batch from $t = 0$ on identical conditions at each Δ .

Delayed feed would naturally decrease the total yield, since penicillin production has less time to cultivate within a fixed 230 h batch. To account for this natural yield drop, that would otherwise result in evaluation bias, we report yield change Δ relative to the recipe baseline of the same seed cell to help us to compare the actual behaviour of the policy.

$$\Delta Y_{\text{policy}}(\text{seed}, \Delta) = Y_{\text{policy}}(\text{seed}, \Delta) - Y_{\text{recipe}}(\text{seed}, \Delta). \quad (3.12)$$

Chapter 4

Results and Evaluation

4.1 Single-GP performance (RQ1)

Regarding our first RQ, Table 4.1 shows that even single-GP without explicit temporal information almost matches the best BO baseline in yield (+148.9 kg vs. +158.7 kg) while being significantly more reliable (± 45 kg vs. ± 193.8 kg – note that SD here is the variance around Δ). Over 30 held-out batches of our single-GP without time resulted in *zero* constraint violations, against 4 on the BO baseline, indicating safe controls over the same number of episodes. We also note that we gave the benefit of the doubt to the BO baseline and only cherry-picked the best-performing models. Nevertheless, all of them showed similar yield gain with drastically worse variance and more collapses from both 15 and 100 episodes 4.1. Thus, under the assumption that 5 historical batches are available, or initial exploratory episodes are allowed, we show that MC-PILCO can indeed be utilised for fed-batch processes converging within 10 training episodes, showing usefulness in low-data settings (Fig. 4.1a, Tab. 4.1). Consistently obtaining a higher yield on held-out seeds also indicates a safe and robust control on fed-batch processes, whereas none of the methods benchmarked in [57] achieved it.

Adding time as a GP input consistently and substantially increases yield. On both Mass and Concentration cost functions, the explicitly time-augmented model further obtained higher yield (3951.0 kg vs 3712.3 kg) and plateaued earlier, with the variance band narrowing by episode 10 4.1a. Relative to the recipe, Time-augmentation added +405.7 kg (Mass) and +387.5 kg (Conc), roughly +11% in both cases, compared to +75.3 and +148.9 kg on MC-PILCO without Time (Fig. 4.1a, A.4a). However, we see that this gain came at the cost of a single viscosity violation per cost function within 30 episodes, showing that the higher yield was obtained by operating closer to the collapse

Method	Held-out mean + Δ yield (kg)	SD across seeds ($n=3$)	SD across batches	Violations
Recipe (policy independent)	r=3563.5	—	—	0/10
BO (best of first 15 evaluations)	r+158.7	± 193.8	± 471.5	4/30
BO (best of first 100 evaluations)	r+163.6	± 167.9	± 621.0	6/30
MC-PILCO (no time)	r+148.9	± 45.0	± 325.9	0/30
MC-PILCO (added time)	r+387.5	± 56.5	± 362.7	1/30

Table 4.1: Recipe, BO, and single-GP MC-PILCO. Yield is the mean of per-seed means. SD is reported both across seeds ($n=3$) and across all 30 seed $\times$ batch evaluations.

boundary (Table A.9).

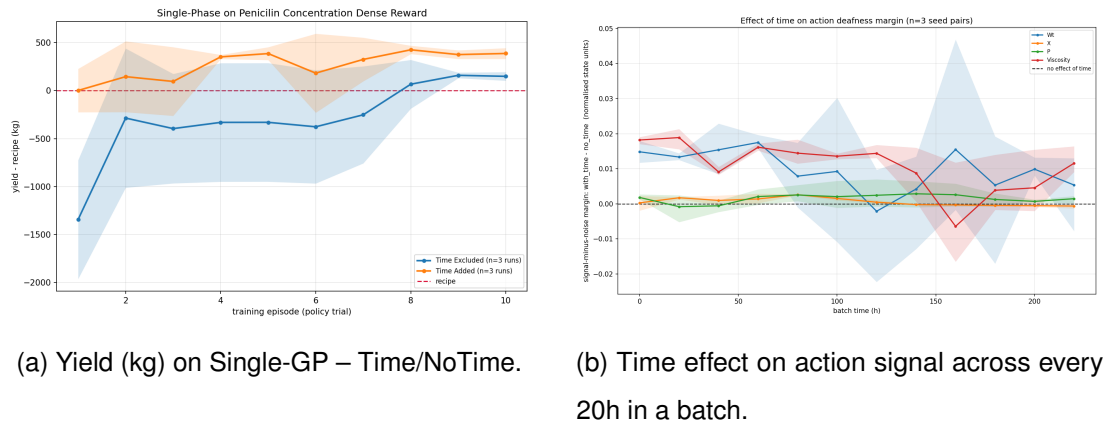


Figure 4.1: Comparing Single-GP models with Time Added vs. Time Excluded.

Despite the higher yield, we next compare how the action signal margin changed with the explicit addition of time. Fig. 4.1b shows that adding time increases this margin for Weight and Viscosity until approximately 80 h, after which the variance increases. Across paired seeds, the increase is significant for Viscosity and Weight, with no detectable change for Biomass or Penicillin, while beyond 80 h neither channel remains significant, suggesting the benefit of explicit time is primarily an early-batch effect.

Therefore, yield improvement appears to arise mainly from better modelling of Weight and Viscosity rather than from a stronger action signal in Biomass or Penicillin. The two channels improve for different reasons. Weight accumulates according to the net flow into and out of the reactor, so it already contains the history of the feed action. Viscosity, instead, has an explicitly time-dependent component in the process model,

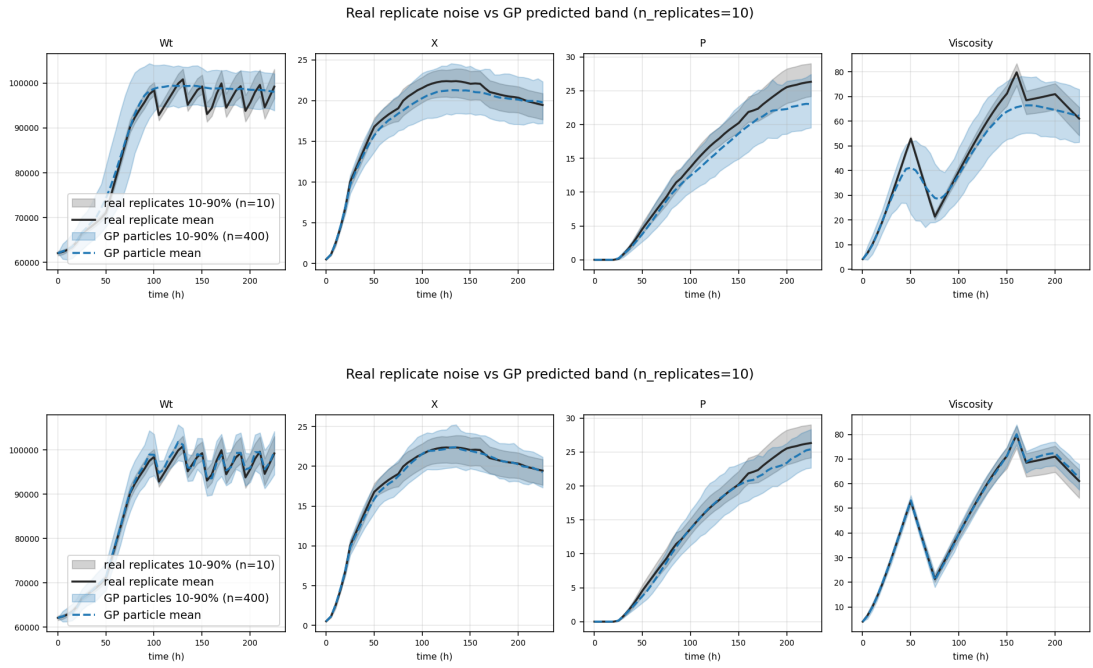


Figure 4.2: GP uncertainty on *noised-out* state trajectories, on single-GP. No Time (above), Time Added (below).

which the GP can only infer indirectly from the observed state when time is excluded, increasing uncertainty around the action effect. This suggests that explicit time captures a predictable early drift but becomes less informative and thus diminishes once the dynamics turn non-stationary.

We further investigate how the addition of time changes the GP model prediction and the uncertainty-covered regions. In Figure 4.2, on the penicillin concentration cost function (also confirmed on mass change cost A.10) we see that adding time also improves how truthful the model is about its own uncertainty. Without time, the GP is under-confident on every channel, with wider confidence bands (for X/P – $1.68/2.23 \times$ the real replicate spread (A.4)).

Adding time also results in better-calibrated state channels (for X/P – $1.34/1.31 \times$ the real replicate spread (A.4)). Biomass and Penicillin also become better calibrated, even though their action signal did not improve earlier (Fig. 4.1b). We observe that *time* makes the state space model’s uncertainty more truthful without making the action’s effect on these two channels any easier to identify, which locates the limitation somewhere other than the uncertainty estimates. We return to that limitation in § 4.2.

Finally, we simulate feed-delay by shifting the measured phase transition later in the batch, from 63 h to 132 h (according to the biomass pivot proxy), which desynchronises

the global time from the culture’s physiological state. Time is not a general solution for representing within-episode changing dynamics, and Table 4.2 exposes the main limitation of this approach, showing that neither model successfully learns the transition. The advantage of explicit time over the no-time model decreases steadily across the 0–60 h range, from +239 kg at no delay to +37 kg at 60 h (although rises again at 80 h, we attribute this to the no-time model becoming highly unstable at this delay with variance of 135 ± 228 kg, larger than the mean, rather model recovery). As the delay increases, both models degrade in two distinct ways. Model *with time* maintains a low variance, losing yield advantage in a consistent, reproducible way rather than behaving robustly. The model with *no time* diverges across seeds and increases in variance, since it lacks explicit temporal information that would represent changing dynamics. Therefore, explicit time acts as a proxy for phase timing rather than a learned representation of transition, and cannot represent within-episode non-stationarity once the transition time between phases varies. Adding time nonetheless remains useful in practice, as at every tested delay it exceeds both the recipe and the no-time model. It also degrades gracefully, so the yield lost under disturbance is predictable rather than erratic. Explicit time is therefore a cheap and safe addition for processes whose transition timing is reliably repeatable [19], but not a general solution where that timing varies.

Feed delay Δ (h)	0	20	40	60	80
Single-GP, time added	388 ± 57	365 ± 148	309 ± 82	255 ± 75	268 ± 26
Single-GP, no time	149 ± 45	221 ± 156	239 ± 212	218 ± 222	135 ± 228
Advantage of time	+239	+144	+70	+37	+133
Recipe yield (kg)	3563	3361	3141	2908	2620

Table 4.2: Yield gain over the recipe (kg, mean $\pm$ SD, 3×10 held-out batches per delay) under the feed-delay disturbance. Advantage indicates difference between models.

We note that the observed yield gain, despite a weak per-step action signal, likely arises from the policy exploiting headroom in the weight and viscosity constraints rather than from biological control. This is indirectly supported by our constraint-strength results (Table A.5), where weight increased from 100.2 t to 108.5 t and peak viscosity from 88 cP to 92 cP, indicating that additional yield is captured volumetrically rather than through higher biological productivity. We also see that using the same policy in an open-loop manner (no feedback), reproduces this yield and variance (Table A.3),

so the controller acts as a learned feed schedule rather than through active biological feedback. Regarding RQ1, MC-PILCO therefore reliably captures the accessible yield in this action space, but that yield is dominated by volumetric rather than biological control, motivating the addition of further action channels (§ 5.2).

4.2 Phase decomposition (RQ2)

4.2.1 Dual-GP Decomposition Results

In single-phase, explicit time raises yield (+387.5 kg), but the feed-delay experiment (Table 4.2) shows this relies on repeatable phase timing, so it acts as a proxy for batch progress rather than learned dynamics. Here, we aim to understand whether supplying the phase structure explicitly, as two independently trained GPs blended across the transition, represents the within-episode non-stationarity better than a single-GP.

We see that under otherwise identical conditions, splitting phases in a fixed morphological manner (T1) increased variance without adding usable control, and failed to reliably improve upon both single-GP variants. We further note that explicit time contributed $\approx +92$ kg (+53 kg and -39 kg Fig. 4.3a) compared to $\approx +239$ kg in a single-GP, and only increased episode-to-episode fluctuation. One interpretation is that the policy now carries three overlapping sources of temporal information, that is the recipe-driven action, the phase blend, and the explicit GP input, so time becomes redundant rather than complementary.

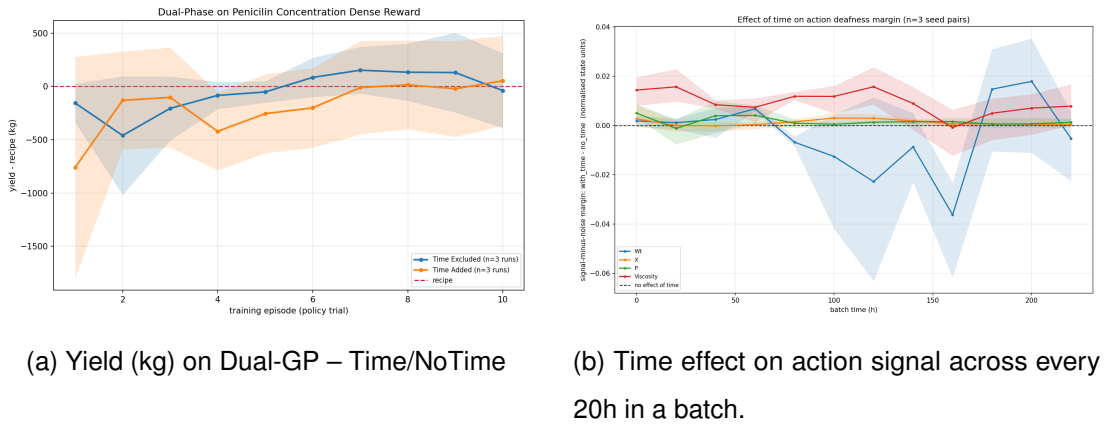


Figure 4.3: Comparing Dual-GP models with Time Added vs. Time Excluded.

Furthermore, we have evaluated different types of transitions, including fixed using morphological variables (T1), and dynamical using biomass and weight as a CER

proxy (T2), applied on each training episode. This dynamical transition is further extended on each rollout (T3), so that each training and imaginary trajectory has different phase split according to that batch's state. Feed-delay experiment is conducted to see if state-adaptive split outperforms the fixed one. Table A.6 indicates that no phase decomposition generalised. In almost all cases, the variance exceeds mean gain, and none indicate a reliable control. Only the T3 turns positive a few times, although these gains are not robust across delay conditions and are accompanied by high variance and highest constraint-violation rate. This means that regardless of the transition boundary, policies fail to generalise, suggesting that multi-phase does not recover reliable control under neither feed delay value. We hypothesise that zero violations are not indication of more reliable policy, but rather that delaying the nutritional feed lowers cumulative substrate input, and hence biomass, making the viscosity not approach its limit.

4.2.2 Relaxing the data and prior assumptions

We further investigate adaptations to multi-phase targeting potential failure modes (Table 4.3). Each variant deliberately relaxes our limited data constraint and prior knowledge assumptions to test whether any of them limits the performance. **More initial exploration** episodes were intended to provide each GP additional training data, motivated by the hypothesis that splitting the process into two phase-specific GPs reduces the number of observations available to each model, potentially degrading individual GP performance [49]. **Extreme feeding schedules (probes)** perturbed the feed during exploration, aiming to increase input variability in under-sampled regions [40]. Finally, we used **recipe-mean GP priors** with the prior mean centred on the recipe trajectory (replacing the zero prior mean of Section 2.3, so the GP models the residual around the recipe trajectory), to stabilise the GP in sparsely sampled regions. We hypothesise that the SE kernel's prior structure may be insufficient to match the distribution of the state channels under the limited data and feed-variance conditions [44]. Our findings suggest that none of the above reliably improved yield (Fig. A.11). More explorations, however, resulted in lower variance and more consistent yield than the rest of the multi-phase methods, explaining that dual-GP underperforms single-GP, due to data fragmentation reducing available training data for each GP.

Priors, however, in the delayed-feed experiment show high relative yield with little variance at no/little delay. Δ Yield gain over the recipe then generally decays as delay increases (+102 kg at 80 h), with increasing variance. This suggests that the priors help

the GP model to learn a time-embedded recipe as a proxy rather than the underlying physical dynamics, which is a similar observation seen with the single-phase with time model. Also, because GP encodes the complete trajectory’s mean, it overstates the progress of the batch, resulting in a few violations for 20 and 40 h delays.

Nevertheless, although we track data limitation as a result of why two GPs performed worse than a single GP, none of the methods address the *fundamental* limitation of multi-phase GP decomposition. Experiments suggest that data decrease, observability and kernel may not be the main bottlenecks of multi-phase design. We therefore investigate the process itself.

Feed delay Δ (h)	0	20	40	60	80
10 exploration episodes	96 ± 197	316 ± 120	296 ± 211	263 ± 209	203 ± 155
Recipe-mean GP priors	279 ± 32	351 ± 89	184 ± 152	126 ± 153	102 ± 156
Feeding probes	46 ± 105	209 ± 157	233 ± 138	220 ± 97	187 ± 52
Violations, 10 expl	1/30	0/30	0/30	0/30	0/30
Violations, priors	1/30	0/30	1/30	1/30	0/30
Violations, probes	1/30	0/30	0/30	0/30	0/30
Recipe yield (kg)	3563	3361	3141	2908	2620

Table 4.3: Yield gain over the recipe baseline (kg, mean $\pm$ SD over three seeds, 30 held-out batches per delay) for the additional multi-phase variants, all using the fixed-pivot split.

4.2.3 Assessing Process Failure: Single-Step Feed Effects and Batch Variation

From the sensitivity analysis in Table 4 of Goldrick et al. [18], the maximum specific production rate ($\mu_{P_{\max}}$) is the highest ranking parameter for Penicillin ($\delta_{msqr} = 0.83$), ahead of the substrate inlet concentration c_s ($\delta_{msqr} = 0.41$), which shares F_s input channel via $F_s c_s / V$ (Eq. 7). $\mu_{P_{\max}}$ is described as a batch-specific value (§ 5 [18]), varying by 0.0449 ± 0.00389 , a 8.7% coefficient of variation across ten industrial batches (Table 3). This suggests that penicillin variation is dominated by an uncontrolled batch-level parameter rather than the feed. Thus we measure the single-step feed effect directly by injecting F_s and differencing it under a paired-seed control, which bounds what GP can identify from a single transition. We see that across our action range, the

entire reachable span of final P stays within 0.48 of a natural inter-episode variation at every injection time (Table 4.4). In the growth phase, every magnitude in both directions reduces yield, so the action carries no usable direction, whereas in production the response orders correctly at 90 and 130 h, but has decayed and changed sign by 170 h. These results suggest that GP cannot reliably identify the action against the inter-batch variation, under limited training episodes.

Injection time	−50%	−25%	+25%	+50%	Range (batch-SD)
30 h	−0.07	−0.01	−0.09	−0.29	0.28
50 h	−0.14	−0.03	−0.18	−0.51	0.48
70 h	−0.21	−0.07	−0.17	−0.53	0.46
90 h	−0.28	−0.13	+0.11	+0.18	0.46
130 h	−0.06	−0.14	+0.12	+0.20	0.34
170 h	−0.27	−0.11	+0.04	−0.02	0.31

Table 4.4: Paired-seed change in final penicillin concentration (P) following feed perturbations at six action injection times, expressed as fractions of the natural inter-batch SD (2.67 g/L, Table A.1) Note – injection times index the substrate operating point on the production curve, not the fixed morphological GP split (§ 3.3).

The production RMSE/SD near 1.0 (Table 4.5) is similar to the size of the signal itself, and since production has more training points than growth, this reflects a feed response that is small relative to inter-batch variation rather than insufficient data points.

State change	n	Target SD	RMSE	RMSE/SD	R^2
Phase 1: ΔX	13	0.283	0.030	0.11	0.989
Phase 2: ΔX	32	0.052	0.049	0.95	0.099
Phase 1: ΔP	13	0.753	0.051	0.07	0.995
Phase 2: ΔP	32	0.065	0.062	0.94	0.115

Table 4.5: One-step GP prediction performance for biomass and penicillin state changes.

4.2.4 Biological Interpretation

Because our policy only makes bounded corrections around the recipe (§ 3.1.2), the GP does not need to learn the full relationship between feed and penicillin production. Instead, it only needs to learn whether deviations from the recipe meaningfully affect the

final yield. Our results in Table 4.5 suggest that the GP in the production phase cannot accurately predict the dynamics model. While Table 4.4 indicates that the action's effect is weak in both phases. The latter is consistent with the simulator's biology, which indicates that the pathways through which feed affects penicillin are shaped differently around the recipe's operating point in each phase. In our case, we identified two: one directly through substrate and another indirectly through biomass mediator. Applying Eq. 11 at these operating points (Tab. A.11) reproduces the measured feed response across both phases, confirming the effect follows from the operating point on the substrate rather than a modelling limitation.

First is the **direct path**, where the sugar feed F_s changes the substrate concentration s , which then affects the penicillin production rate μ_P . The production rate μ_P is a unimodal Gaussian function of substrate, with a peak at $s_{P_{\max}}$ ([18], Eq. 11). Too little sugar limits production, while too much can suppress production through carbon catabolite repression [39].

During *growth*, the recipe keeps the substrate near the peak ($z \approx +0.29$, $\approx 96\%$ of maximum), where the first-order gradient vanishes (see A.1). The maximum possible gain from any substrate move is therefore only $\approx +4\%$ (the distance to the peak), while moving further out is unbounded, resulting in asymmetry. This is consistent with Table 4.4, where any perturbation during growth reduces final yield. During *production*, the recipe moves the substrate to the rising side of the Gaussian ($z \approx -0.6$ to -0.9). Here, substrate is consumed rapidly (Eq. 7, [18]), quickly depleting the existing substrate. As a result, the substrate concentration is mainly controlled by the sugar feed. The effect of F_s on μ_P depends on how strongly the feed changes substrate concentration ($\partial s / \partial F_s$), and how strongly substrate affects the production rate ($\partial \mu_P / \partial s$). Using the chain rule, $\partial \mu_P / \partial F_s = (\partial \mu_P / \partial s)(\partial s / \partial F_s)$ (Eq. 11, 7). In production, both terms are strong, so a single feed step can affect the production rate. However, the resulting change in final yield is small relative to inter-batch variation, making the effect difficult to identify at each step. In growth, by contrast, the direct feed effect is capped by the asymmetry above, remaining weak overall (Table 4.4).

The second is the **indirect path** through the producing biomass A_1 . Feed affects A_1 , which in turn affects penicillin production over the remaining production horizon ([18], Eqs. 2, 6). During *growth*, a feed increase has two opposing effects. It increases the A_1 , which can increase final yield, but it also directly reduces production through the substrate effect described above. The direct effect is stronger, so the net effect is a reduction in final yield. Here, this effect is also limited by viscosity, which is

driven by the growing region A_0 (Eq. 28, [18]) and is highest during this phase. During *production*, the direct and biomass effects act in the same direction, so additional feed raises final yield monotonically, whereas during growth the direct penalty outweighs the biomass gain. A feed increase produces a similar amount of additional biomass at harvest in both phases, which gives production a persistent feed signal. However, exploiting this effect requires sustained feed, which pushes the batch toward the weight constraint rather than increasing the production rate. When we allowed the action to move further from the recipe to exploit this effect, roughly a third of batches violated constraints without achieving a net gain (Fig. A.8). Therefore, the production feed signal is real but difficult to exploit safely.

This suggests a limitation in control rather than observability. To test whether the weak feed signal was hidden by insufficient state information, we augmented the GP inputs with internal simulator variables, including substrate, dissolved oxygen, and the growing/producing biomass regions affected by the feed. As per Table A.10, it did not recover a stronger signal, suggesting that the weak feed effect is not caused by hidden states. Therefore, we interpret the limitation as weak control authority rather than partial observability.

4.2.5 Remarks on single and dual-GP decomposition

Phase decomposition does not improve control because neither phase provides an action response that is sufficiently large relative to the noise resulting from batch-to-batch variation. Our biological analysis shows that growth and production differ in how feed affects the system, but these differences do not translate into stronger actionable signals within the restricted action space. The single GP already performs well by following the recipe’s near-optimal feed profile while managing biomass and constraints. Decomposition therefore does not provide an additional control signal to exploit, regardless of the phase pivot or additional training. This suggests that the main limitation is the control authority of the action space rather than the phase decomposition itself. Due to this weak relative action signal, phase-specific models have no additional signal to exploit, agreeing with batch-RL literature, which identifies run-to-run variability as the main challenge for these processes [54].

Chapter 5

Conclusion and Discussion

In this dissertation, we explored whether MC-PILCO, a model-based GP policy-search algorithm designed for stationary robotic control, can be adapted to a non-stationary, high-variability fed-batch bioprocess, and whether injecting the known growth/production phase structures as two phase-specific GP dynamics models addresses intra-episode non-stationarity better than a single GP.

RQ1 – Adapting MC-PILCO to fed-batch fermentation.

In this dissertation, we saw that MC-PILCO transfers successfully, which, to our knowledge, is the first PILCO-family application to fed-batch processes. From five explorations and ten policy-search episodes, a single-GP model without explicit time almost matches the BO baseline in yield (+148.9 vs. +158.7 kg over the recipe) while achieving gains more reliably (± 45 across-seed spread vs. BO's ± 193.8 kg) with no constraint violations across 30 held-out batches, against BO's 4–6. Compared to Zhang et al. [57] methods, where none outperformed the BO baseline, matching its yield with lower variance and no violations shows that MC-PILCO can be a reliable and data-efficient alternative, although we note this is due to better Viscosity and Weight modeling.

Adding explicit time as a GP input substantially raised the yield to +387.5 kg over the recipe. However, it resulted in one viscosity violation, indicating that the policy exploits actions closer to the collapse boundary, rather than better modelling of penicillin concentration and biomass. Furthermore, under a simulated feed-delay, time's advantage decreases, although it still beats both the recipe and no-time model at every delay tested. Therefore, explicit time acts as a proxy for learning system biological dynamics, but does not actually capture them. Explicit time remains a viable solution when phase timing is repeatable, but not a general solution for intra-episode

non-stationarity when transition time varies. This inspired the search for biologically motivated model alternatives explored in our second research question.

RQ2 – Does a phase-aware dual-GP improve on a single GP?

We found that regardless of the type and location of the transition pivot, no dual-GP decomposition performed more reliably than the single-GP baselines, as indicated by higher variance and lower yield. Through targeted experiments addressing data limitations, action exploration coverage, and kernel priors, we saw that the phase split limits the data for each GP, respectively, likely accounting for the increase in variance and leads to lower yield compared to single-GP.

Our analysis showed that the main limitation, however, is high batch-to-batch variability and noise, which diminishes the substrate feed action influence. The model’s success is reliant on its ability to detect small fluctuations in residual values, and the noise inhibits its ability to do so, leaving it unable to optimise above the recipe. We investigated the substrate-inhibition curve of the simulator and saw that the action could not gain precedence over the noise in either phase, for different reasons – the growth phase action signal was strongly asymmetric near the peak, limiting the achievable gain, whereas in the production phase, the action signal was below the inter-batch variability. Our single-GP open-loop experiments showed that the policy’s feedback loop barely contributes to yield gain (A.3), and the policy acts as a learned feed schedule. And when the action was allowed to deviate freely from the recipe, roughly one-third of batches violated constraints without net gain (A.8), confirming that the sugar feed offers no safely exploitable authority in this data-limited setting.

Thus, we saw that neither explicit time nor dual-GP is a general solution for intra-episode non-stationarity when controlling only a single feed channel over the already optimised recipe, as none of the methods helped to overcome the action signal problem. We note that because of this, our previously mentioned phase decomposition idea, where each state variable would have its own transition time and blend width, is unlikely to improve control in this action space and could only compound the data-fragmentation cost of splitting the GPs.

5.1 Generalisability

The MC-PILCO adaptation itself requires only a nominal recipe and an observable or soft-sensed state, but the phase decomposition additionally requires sufficient action authority relative to batch-to-batch variation. This latter condition, however, failed in

our IndPenSim experimentation, as the feed-action signal was absorbed within one inter-batch deviation. We expect this approach to be transferable for processes where feed has a stronger and more identifiable effect, for example, by crossing a threshold in specific substrate uptake, as in acetate overflow in *E. coli* [52, 1], or where the phase boundary is directly specified by operation, as in the glycerol-to-methanol switch in *P. pastoris* [58]. We note that these are our expectations only, and establishing whether the approach transfers would require measuring the effect-to-variability ratio directly in each process, which we leave to future work.

These findings are also specific to residual corrections around an already near-optimal industrial recipe. They should therefore not be interpreted as a general claim about the usefulness of substrate feed, which may have substantially greater signal in a broader action space, or a different fed-batch simulator. Thus we suggest, before selecting a dynamics-model architecture, perturbing the candidate action under paired seeds and expressing the resulting objective span relative to the inter-batch standard deviation. Where the action-induced span is small relative to this variation, the main limitation is the identifiability of the action rather than the representation of the dynamics. Further phase decomposition, kernel changes, or additional state inputs are then unlikely to recover the missing control signal. In such cases, effort should instead focus on enriching the action space.

5.2 Limitations and Future Work

To improve the controls reliably and safely, and address data efficiency concerns, we suggest additional action-space extensions rather than fixing the dynamics-model architecture itself. Such actions may include aeration (F_g) or agitation (RPM) controls, which influence penicillin production through dissolved-oxygen availability and oxygen transfer rather than substrate (s) alone. This opens more controls that the sugar feed could not reach.

Alternate approaches include using different action designs per phase: one recipe-driven control in the growth phase, and in the production phase, the policy could explore more freely, beyond the recipe. This way, the policy in the growth phase would learn reliable controls, whereas in production, it could learn a higher action signal. Future work should also train the models on more than $n = 3$ seeds, or use different simulators for other microorganisms cultivated using fed-batch processes, to prove method transferability, for example, from *Danu Insights* [10].

Bibliography

- [1] Merouane Abadli, Laurent Dewasme, Sihem Tebbani, Didier Dumur, and Alain Vande Wouwer. Experimental validation of a nonlinear model predictive controller regulating the acetate concentration in fed-batch escherichia coli bl21 (de3) cultures. *Advanced Control for Applications: Engineering and Industrial Systems*, 4 (1):e95, 2022.
- [2] Lucas N Alegre, Ana LC Bazzan, and Bruno C da Silva. Minimum-delay adaptation in non-stationary reinforcement learning via online high-confidence change-point detection. *arXiv preprint arXiv:2105.09452*, 2021.
- [3] Fabio Amadio, Alberto Dalla Libera, Riccardo Antonello, Daniel Nikovski, Ruggero Carli, and Diego Romeres. Model-based policy search using monte carlo gradient estimation with real systems application. *IEEE Transactions on Robotics*, 38(6):3879–3898, 2022. doi: 10.1109/TRO.2022.3184837.
- [4] Ahmad Ashoori, Behzad Moshiri, Ali Khaki-Sedigh, and Mohammad Reza Bakhtiari. Optimal control of a nonlinear fed-batch fermentation process using model predictive approach. *Journal of Process Control*, 19(7):1162–1173, 2009.
- [5] RK Bajpai and M Reuss. A mechanistic model for penicillin production. *Journal of Chemical Technology and Biotechnology*, 30(1):332–344, 1980. doi: 10.1002/jctb.503300140.
- [6] Theodora A Bibila and David K Robinson. In pursuit of the optimal fed-batch process for monoclonal antibody production. *Biotechnology progress*, 11(1):1–13, 1995.
- [7] Annie S Booth, Andrew Cooper, and Robert B Gramacy. Nonstationary gaussian process surrogates. *arXiv preprint arXiv:2305.19242*, 2023.

- [8] Konstantinos Chatzilygeroudis, Roberto Rama, Rituraj Kaushik, Dorian Goepp, Vassilis Vassiliades, and Jean-Baptiste Mouret. Black-box data-efficient policy search for robotics. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 51–58. IEEE, 2017.
- [9] Steven J Coppella and Prasad Dhurjati. A detailed analysis of *saccharomyces cerevisiae* growth kinetics in batch, fed-batch, and hollow-fiber bioreactors. *The Chemical Engineering Journal*, 41(2):B27–B35, 1989.
- [10] Danu Insights. <https://www.danu-insights.com/>, 2026. Accessed: 2026-08-24.
- [11] Marc Deisenroth and Carl E Rasmussen. Pilco: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on machine learning (ICML-11)*, pages 465–472, 2011. URL <https://icml.cc/2011/papers/323;cmlpaper.pdf>.
- [12] Rutger D. Douma. Regulation, transport aspects and degeneration of penicillin biosynthesis in *penicillium chrysogenum*. 2010. URL <https://api.semanticscholar.org/CorpusID:85866943>.
- [13] Sylvia Y Edifor, Quoc D Nguyen, Philip Van Eyk, Patrick Biller, Tony Hall, and David M Lewis. Viscosity variation of model compounds during hydrothermal liquefaction under subcritical conditions of water. *Industrial & Engineering Chemistry Research*, 60(2):980–989, 2021.
- [14] RP Elander. Industrial production of β -lactam antibiotics. *Applied microbiology and biotechnology*, 61(5):385–392, 2003.
- [15] Fan Feng, Biwei Huang, Kun Zhang, and Sara Magliacane. Factored adaptation for non-stationary reinforcement learning. *Advances in Neural Information Processing Systems*, 35:31957–31971, 2022.
- [16] Yarin Gal, Rowan McAllister, and Carl Edward Rasmussen. Improving pilco with bayesian neural network dynamics models. In *Data-efficient machine learning workshop, ICML*, volume 4, page 25, 2016.
- [17] Aydin Golabgir and Christoph Herwig. Combining mechanistic modeling and raman spectroscopy for real-time monitoring of fed-batch penicillin production. *Chemie Ingenieur Technik*, 88(6):764–776, 2016.

- [18] Stephen Goldrick, Andrei Ștefan, David Lovett, Gary Montague, and Barry Lennox. The development of an industrial-scale fed-batch fermentation simulation. *Journal of biotechnology*, 193:70–82, 2015. doi: 10.1016/j.jbiotec.2014.10.029.
- [19] Stephen Goldrick, Carlos A Duran-Villalobos, Karolis Jankauskas, David Lovett, Suzanne S Farid, and Barry Lennox. Modern day monitoring and control challenges outlined on an industrial-scale benchmark fermentation process. *Computers & Chemical Engineering*, 130:106471, 2019. doi: 10.1016/j.compchemeng.2019.05.037.
- [20] Klemens Iten, Bruce Lee, Chenhao Li, Lenart Treven, Andreas Krause, and Bhavya Sukhija. Model-based reinforcement learning for control under time-varying dynamics. *arXiv preprint arXiv:2604.02260*, 2026.
- [21] Qingchao Jiang, Yichao Yang, Guan Wang, Zhenhua Yu, Xuefeng Yan, and Weimin Zhong. Bomp-gam: A stage-aware and interpretable soft sensing framework for nonstationary biomanufacturing processes. *IEEE Transactions on Industrial Informatics*, 2026.
- [22] Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. Residual reinforcement learning for robot control. In *2019 international conference on robotics and automation (ICRA)*, pages 6023–6029. IEEE, 2019.
- [23] Julian Kager, Andrea Tuveri, Sophia Ulonska, Paul Kroll, and Christoph Herwig. Experimental verification and comparison of model predictive, pid and model inversion control in a penicillium chrysogenum fed-batch process. *Process biochemistry*, 90:1–11, 2020. doi: 10.1016/j.procbio.2019.11.023.
- [24] Jong Woo Kim, Byung Jun Park, Tae Hoon Oh, and Jong Min Lee. Model-based reinforcement learning and predictive control for two-stage optimal control of fed-batch bioreactor. *Computers & Chemical Engineering*, 154:107465, 2021. doi: 10.1016/j.compchemeng.2021.107465.
- [25] Martin Kluge, Detlef Siegmund, Hans Diekmann, and Manfred Thoma. A model for penicillin production with and without temperature shift after the growth phase. *Applied microbiology and biotechnology*, 36(4):446–451, 1992. doi: 10.1007/BF00170180.

- [26] Jeongseok Lee, Sang Yup Lee, Sunwon Park, and Anton PJ Middelberg. Control of fed-batch fermentations. *Biotechnology advances*, 17(1):29–48, 1999.
- [27] Haoran Li, Tong Qiu, and Fengqi You. Ai-based optimal control of fed-batch biopharmaceutical process leveraging deep reinforcement learning. *Chemical Engineering Science*, 292:119990, 2024. doi: 10.1016/j.ces.2024.119990.
- [28] Henry C. Lim and Hwa Sung Shin. *Introduction to Fed-Batch Cultures*, page 1–18. Cambridge Series in Chemical Engineering. Cambridge University Press, 2013. doi: 10.1017/CBO9781139018777.
- [29] Reiner Luttmann, Daniel G Bracewell, Gesine Cornelissen, Krist V Gernaey, Jarka Glassey, Volker C Hass, Christian Kaiser, Christian Preusse, Gerald Striedner, and Carl-Fredrik Mandenius. Soft sensors in bioprocessing: a status report and recommendations, 2012.
- [30] Thomas M. Moerland, Joost Broekens, Aske Plaat, and Catholijn M. Jonker. Model-based reinforcement learning: A survey. *Foundations and Trends in Machine Learning*, 16(1):1–118, 2023.
- [31] Lisa Mears, Stuart M Stocks, Gürkan Sin, and Krist V Gernaey. A review of control strategies for manipulating the feed rate in fed-batch fermentation processes. *Journal of biotechnology*, 245:34–46, 2017.
- [32] GA Montague, AJ Morris, AR Wright, M Aynsley, and A Ward. Parameter adaptive control of the fed-batch penicillin fermentation. *IFAC Proceedings Volumes*, 18(17):69–74, 1985.
- [33] Max Mowbray, Panagiotis Petsagkourakis, Ehecatl Antonio del Rio-Chanona, and Dongda Zhang. Safe chance constrained reinforcement learning for batch process control. *Computers & chemical engineering*, 157:107630, 2022.
- [34] Anusha Nagabandi, Chelsea Finn, and Sergey Levine. Deep online learning via meta-learning: Continual adaptation for model-based rl. *arXiv preprint arXiv:1812.07671*, 2018.
- [35] Paavo Parmas, Carl Edward Rasmussen, Jan Peters, and Kenji Doya. Pippis: Flexible model-based policy search robust to the curse of chaos. In *International Conference on Machine Learning*, pages 4065–4074. PMLR, 2018.

- [36] GC Paul and CR Thomas. A structured model for hyphal differentiation and penicillin production using penicillium chrysogenum. *Biotechnology and bioengineering*, 51(5):558–572, 1996. doi: 10.1002/(SICI)1097-0290(19960905)51:5<558::AID-BIT8>3.0.CO;2-B.
- [37] D Picque and Georges Corrieu. New instrument for on-line viscosity measurement of fermentation media. *Biotechnology and bioengineering*, 31(1):19–23, 1988.
- [38] Carl Rasmussen and Zoubin Ghahramani. Infinite mixtures of gaussian process experts. *Advances in neural information processing systems*, 14, 2001.
- [39] GLORIA REVILLA, MANUEL J LÓPEZ-NIETO, JOSE M LUENGO, and JUAN F MARTIN. Carbon catabolite repression of penicillin biosynthesis by penicillium chrysogenum. *The Journal of Antibiotics*, 37(7):781–789, 1984.
- [40] Ryan Roussel, Juan Pablo Gonzalez-Aguilera, Young-Kee Kim, Eric Wisniewski, Wanming Liu, Philippe Piot, John Power, Adi Hanuka, and Auralee Edelen. Turn-key constrained parameter space exploration for particle accelerators using bayesian active learning. *Nature communications*, 12(1):5612, 2021.
- [41] Rosmawati Rumi, Min Keng Tan, Kenneth Tze Kin Teo, Sivakumar Kumaresan, and Heng Jin Tham. Reinforcement learning-based feed rates control for fed-batch penicillin fermentation. In *2025 IEEE International Conference on Artificial Intelligence in Engineering and Technology (IICAJET)*, pages 760–765. IEEE, 2025.
- [42] Dewey DY Ryu and J Hospodka. Quantitative physiology of penicillium chrysogenum in penicillin fermentation. *Biotechnology and Bioengineering*, 22(2):289–298, 1980. doi: 10.1002/bit.260220204.
- [43] Steindór Sæmundsson, Katja Hofmann, and Marc Peter Deisenroth. Meta reinforcement learning with latent variable gaussian processes. *arXiv preprint arXiv:1803.07551*, 2018.
- [44] Daniel Sanz-Alonso and Ruiyi Yang. Gaussian process regression under computational and epistemic misspecification. *SIAM Journal on Numerical Analysis*, 63(2):495–519, 2025.

- [45] Joohyun Shin, Thomas A Badgwell, Kuang-Hung Liu, and Jay H Lee. Reinforcement learning—overview of recent progress and implications for process control. *Computers & Chemical Engineering*, 127:282–294, 2019.
- [46] Gary Simethy. *Implementation and assessment of Advanced Reinforcement Learning methods for control of chemical processes*. PhD thesis, 2024.
- [47] Gary Simethy, Margret Bauer, Ruomu Tan, and Fabian Buelow. Robust predictable control (rpc) for optimizing fed-batch penicillin production. *IFAC-PapersOnLine*, 59(6):385–390, 2025. doi: 10.1016/j.ifacol.2025.07.176.
- [48] Richard S Sutton, Andrew G Barto, and Andrew Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- [49] Nick Terry and Youngjun Choe. Splitting gaussian processes for computationally-efficient regression. *Plos one*, 16(8):e0256470, 2021.
- [50] Nagesh K Tripathi and Ambuj Shrivastava. Scale up of biopharmaceuticals production. In *Nanoscale fabrication, optimization, scale-up and biological aspects of pharmaceutical nanotechnology*, pages 133–172. Elsevier, 2018.
- [51] Christopher KI Williams and Carl Edward Rasmussen. *Gaussian processes for machine learning*, volume 2. MIT press Cambridge, MA, 2006.
- [52] Bo Xu, Mehmedalija Jahic, and Sven-Olof Enfors. Modeling of overflow metabolism in batch and fed-batch cultures of escherichiacoli. *Biotechnology progress*, 15(1):81–90, 1999.
- [53] Yanda Yang and Sambaeta Das. Residual rl-mpc for robust microrobotic cell pushing under time-varying flow. *arXiv preprint arXiv:2603.05448*, 2026.
- [54] Haeun Yoo, Ha Eun Byun, Dongho Han, and Jay H Lee. Reinforcement learning for batch process control: Review and perspectives. *Annual Reviews in Control*, 52:108–119, 2021. doi: 10.1016/j.arcontrol.2021.10.006.
- [55] Haeun Yoo, Boeun Kim, Jong Woo Kim, and Jay H Lee. Reinforcement learning based optimal control of batch processes using monte-carlo deep deterministic policy gradient with phase segmentation. *Computers & Chemical Engineering*, 144:107133, 2021. doi: 10.1016/j.compchemeng.2020.107133.

- [56] Mohan Zhang. PenSimPy: The python implementation of IndPenSim, 2020. URL <https://github.com/Mohan-Zhang-u/PenSimPy>. GitHub repository.
- [57] Mohan Zhang, Xiaozhou Wang, Benjamin Decardi-Nelson, Bo Song, An Zhang, Jinfeng Liu, Sile Tao, Jiayi Cheng, Xiaohong Liu, DengDeng Yu, et al. Smpl: Simulated industrial manufacturing and process control learning environments. *Advances in Neural Information Processing Systems*, 35:26631–26644, 2022. doi: 10.48550/arXiv.2206.08851.
- [58] Wenhui Zhang, Karen J Hywood Potter, Bradley A Plantz, Vicki L Schlegel, Leonard A Smith, and Michael M Meagher. Pichia pastoris fermentation with mixed-feeds of glycerol and methanol: growth kinetics and production improvement. *Journal of Industrial Microbiology and Biotechnology*, 30(4):210–215, 2003.

Appendix A

Supplementary Results

Algorithm 1 simplified high level MC-PILCO [3]

```

1: init policy  $\pi_{\theta}$ , cost  $c$ , kernel  $k$ ,  $N_{opt}$ , number of particles  $M$ , and other learning and
   monitoring signal parameters.
2: Apply exploratory control to system and collect data (§ 3.2.1)
3: while task not learned do
4:   1) Model Learning:
5:   Learn GP models from sampled data;
6:   2) Policy Update:
7:   Initialize monitoring signal  $s_0 = 0$ ;
8:   for  $j = 1 \dots N_{opt}$  do
9:     Simulate  $M$  particles rollouts with GP models and current policy  $\pi_{\theta_j}(\cdot)$ ;
10:    Compute  $\hat{J}(\theta_j)$  from particles;
11:    Compute  $\nabla_{\theta} \hat{J}(\theta_j)$  through backpropagation;
12:    Gradient-based policy update  $\rightarrow \pi_{\theta_{j+1}}(\cdot)$ ;
13:    Update monitoring signal  $s_j$ ;
14:   end for
15:   3) Policy Execution:
16:   apply updated policy to system and collect data
17: end while
18: return trained policy, learned GP model;

```

Listing A.1: MATLAB code from `fctrl_indpensim.m`, lines 255–263 – this procedure is applied to every state channel.

```
Recipe_discharge = [500 510 ... 1360 1750 ];
```

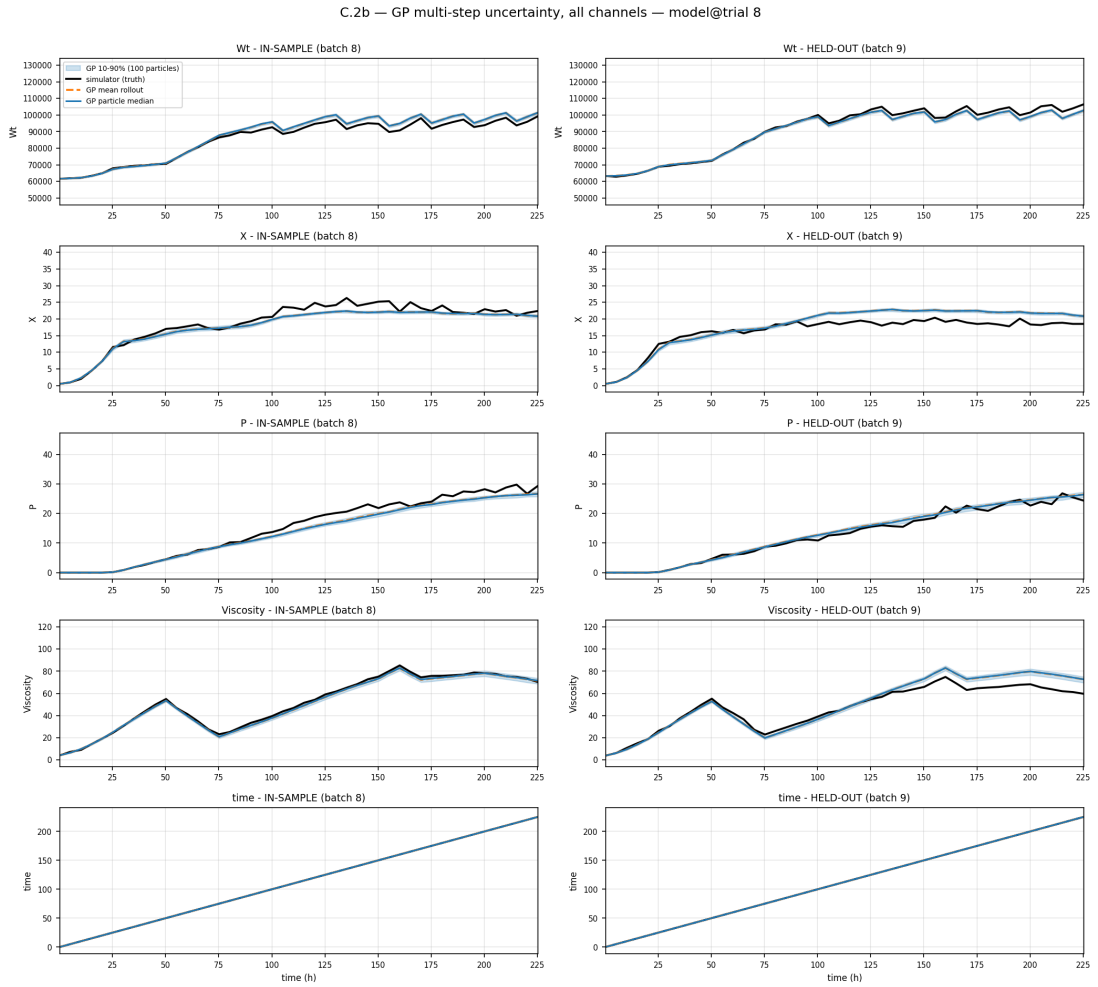


Figure A.1: Narrow GP bands, indicating model is over confident, which is dangerous when environment is intra- and inter-episode non-stationary, with highly stochastic dynamics varying over each seed.

```

Recipe_discharge_sp = [ 0  4000  ...  0  0 ];
for SQ = 1:1:length(Recipe_discharge)
    if k<=Recipe_discharge(SQ)
        F_discharge = -Recipe_discharge_sp(SQ);
        break
    else
        F_discharge = Recipe_discharge_sp(end);
    end
end
end

```

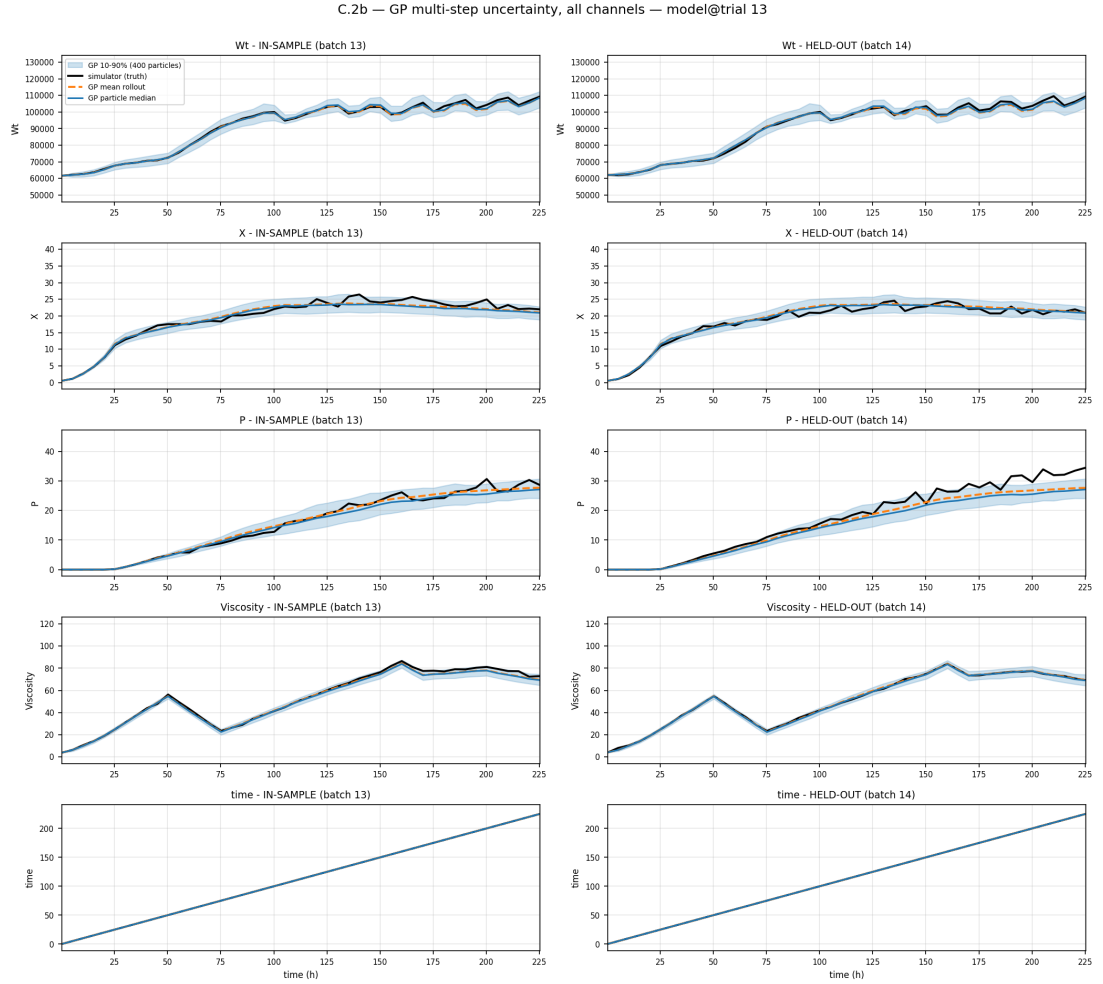
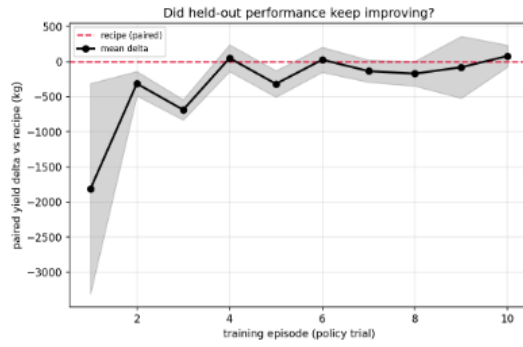
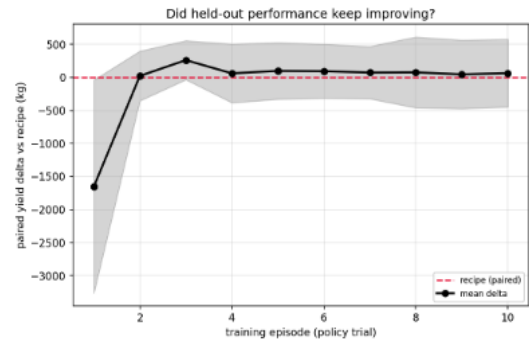


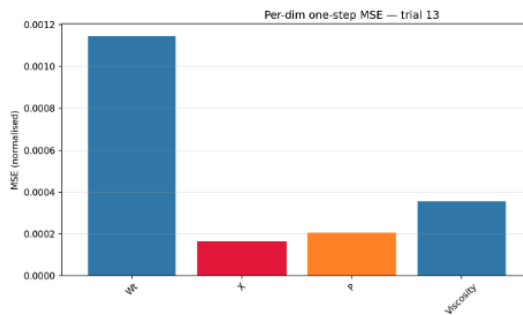
Figure A.2: After the variance fix GP bands are more conservative and less over confident, which is preferred in stochastic fed-batch systems.



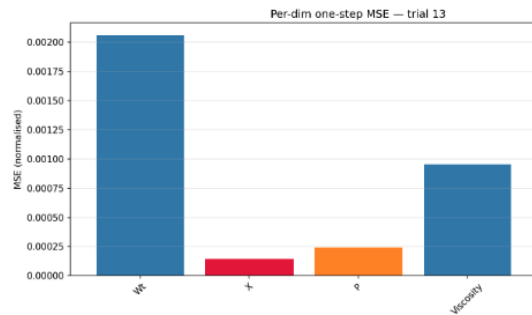
(a) 2 h sampling – improvement



(b) 5 h sampling – improvement



(c) 2 h sampling – MSE



(d) 5 h sampling – MSE

Figure A.3: Comparing Single-phase results for different Step Sizes. Agent acts every 2h vs. 5h, under otherwise identical settings.

Metric	Mean $\pm$ SD	Unit
Penicillin Yield	3444.52 $\pm$ 273.18	kg
Final penicillin concentration	26.71 $\pm$ 2.67	g/L
Maximum penicillin concentration	26.74 $\pm$ 2.64	g/L
Mean penicillin concentration	14.25 $\pm$ 1.25	g/L
Final volume	91747.58 $\pm$ 3166.88	L
Mean pH	6.49 $\pm$ 0.04	–
Mean temperature	298.01 $\pm$ 0.08	K
Final biomass	19.69 $\pm$ 1.44	g/L
Maximum biomass	22.46 $\pm$ 1.69	g/L
Mean biomass	17.99 $\pm$ 1.10	g/L
Final weight	100934.30 $\pm$ 3023.67	kg
Maximum weight	102274.95 $\pm$ 2411.46	kg
Mean weight	88057.38 $\pm$ 1631.09	kg
Maximum viscosity	79.51 $\pm$ 4.09	cP
Mean viscosity	48.93 $\pm$ 2.33	cP

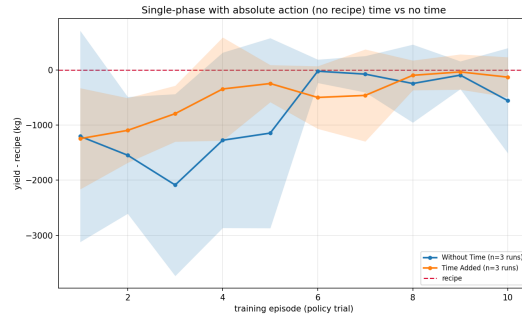
Table A.1: Mean and standard deviation of the main process metrics across 100 batch runs following the default recipe and simulator embedded PID for pH and temperature, on randomly selected seeds.

Model	Concentration cost	Mass cost
SP + time	1	1
SP - time	0	0
MP + time	1	2
MP - time	0	1

Table A.9: Number of constraint violations for each model and cost function across. Tested across 30 batches per model.

<i>Dynamics GPs</i>	
Kernel	SE, ARD, zero mean
Optimiser	Adam, lr 0.01, 500 epochs/trial
Noise	$\sigma_n = 0.01$ (trainable) / 10^{-3}
Sparsification	subset-of-data, threshold 0.3
<i>Policy</i>	
Basis functions	100 (605 parameters)
Dropout	0.25 (optimisation only)
Squashing	$\tanh, u_{\max} = 1$
<i>Policy optimisation</i>	
Particles	400
Horizon	45 steps (5 h decisions)
Optimiser	Adam, lr 0.01, ≤ 1000 steps, plateau schedule
Initial state	$\mathcal{N}$ from recipe batches
<i>Learning loop</i>	
Exploration	5 batches, 12-segment piecewise-constant
Trials	10

Table A.2: Algorithmic settings. State and action design are given in Section 3; the cost is detailed in Section 3.1.4.

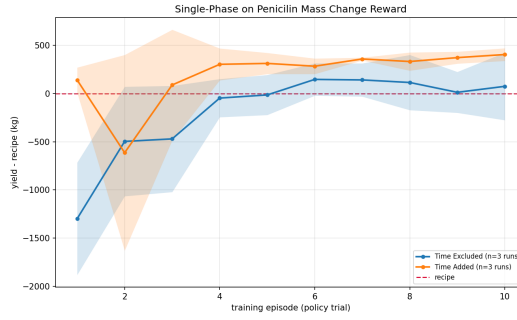


(a) Single phase, where F_s does not follow recipe, and instead $a_t \in [0, 200]$.

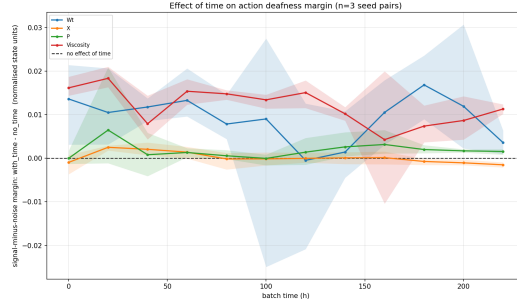
	Added Time (violations)	No Time (violations)	Total violations
Seed 4	3/15	6/15	30%
Seed 5	5/15	6/15	36.7%
Seed 6	2/15	9/15	36.7%

(b) Batch viscosity or weight violations for each training seed (added time vs no time).

Figure A.8: Comparison of absolute-action yield and batch viscosity or weight violations.



(a) Mass Change Reward Function



(b) Single phase Time effect on action signal across every 20h in a batch, across 3 seeds for Mass change cost function

Figure A.4: Comparing Single-GP models with Time Added vs. Time Excluded. Tested on Mass Change cost function.

A.1 Substrate operating point on the production curve

The closed-form production-rate model of Eq. A.1 predicts the measured paired-seed feed response across both phases (Pearson $\rho = 0.91$, slope $k=0.94$, $R^2=0.80$, correct sign in 20/24 cells), so the weak and phase-dependent feed effect follows directly from the operating point on the substrate-inhibition curve rather than from any modeling limitation.

The penicillin production rate depends on the substrate through a Gaussian centered on $s_{P_{\max}}$ (Eq. 11, [18]), implemented with $s_{P_{\max}} = 2.0 \times 10^{-3}$ g/L and $\sigma_P = 1.5 \times 10^{-3}$ g/L. Expressed relative to its own maximum, the amplitude cancels and the operating point depends only on the standardised substrate $z = (s - s_{P_{\max}})/\sigma_P$:

$$\frac{\mu_P(s)}{\mu_P(s_{P_{\max}})} = \exp\left(-\frac{1}{2}z^2\right). \quad (\text{A.1})$$

Taking the median substrate trajectory over ten recipe-driven batches, the growth phase sits just above the optimum at $z = +0.29$ (96% of maximum), where the gradient is

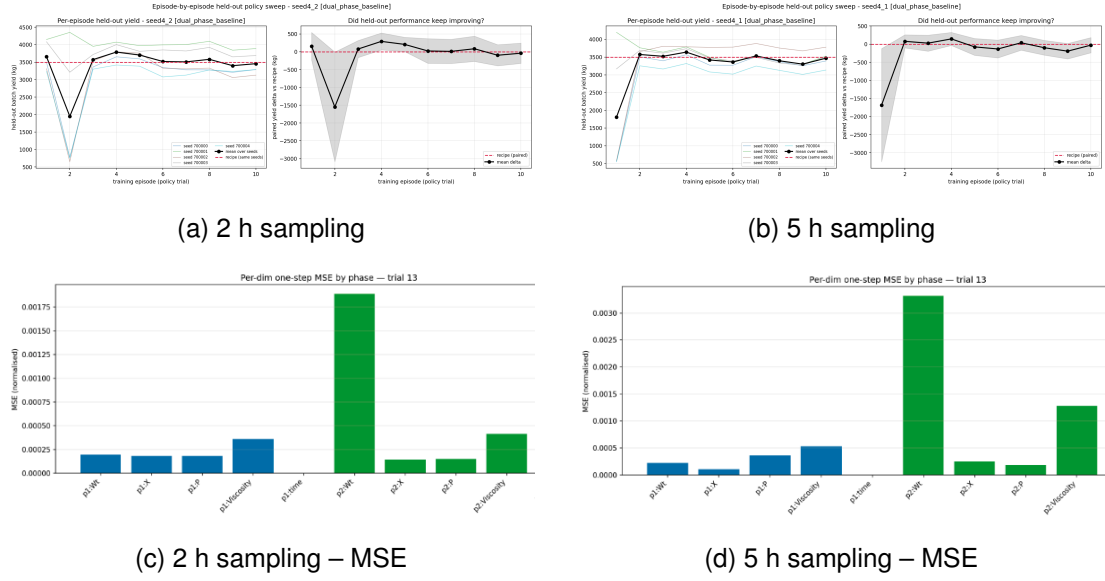


Figure A.5: Comparing Multi-phase results for different Step Sizes. Agent acts every 2h vs. 5h, under otherwise identical settings.

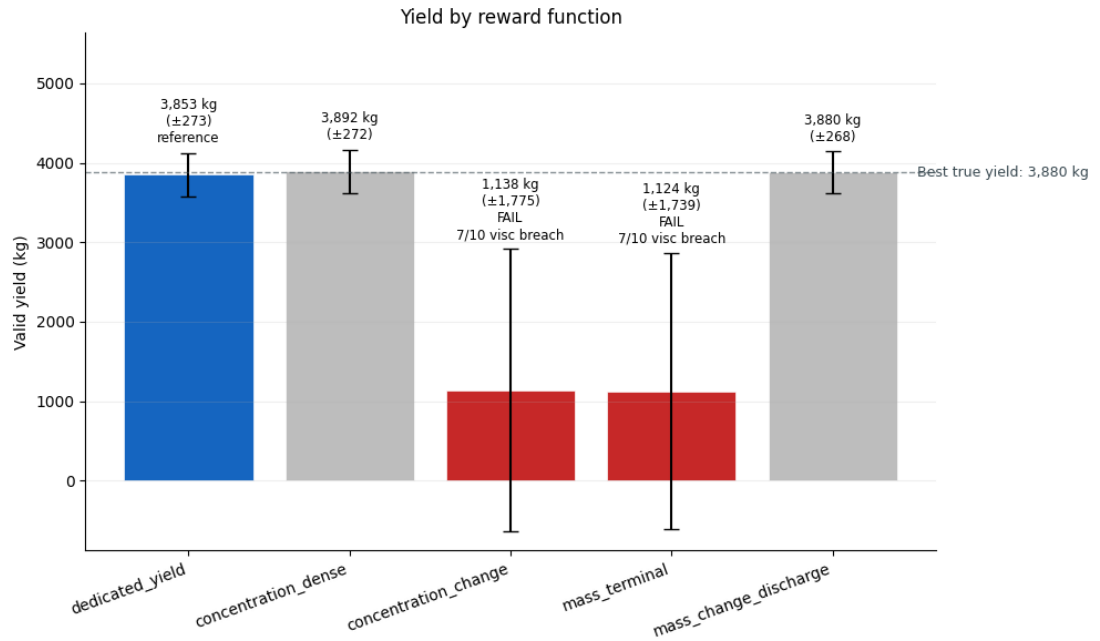


Figure A.6: Reward function tests under Bayesian Optimisation reward hacking. ΔY is largest for Penicillin mass terminal and concentration dense cost functions, thus they were discarded. Lower Δ indicates better alignment with objective.

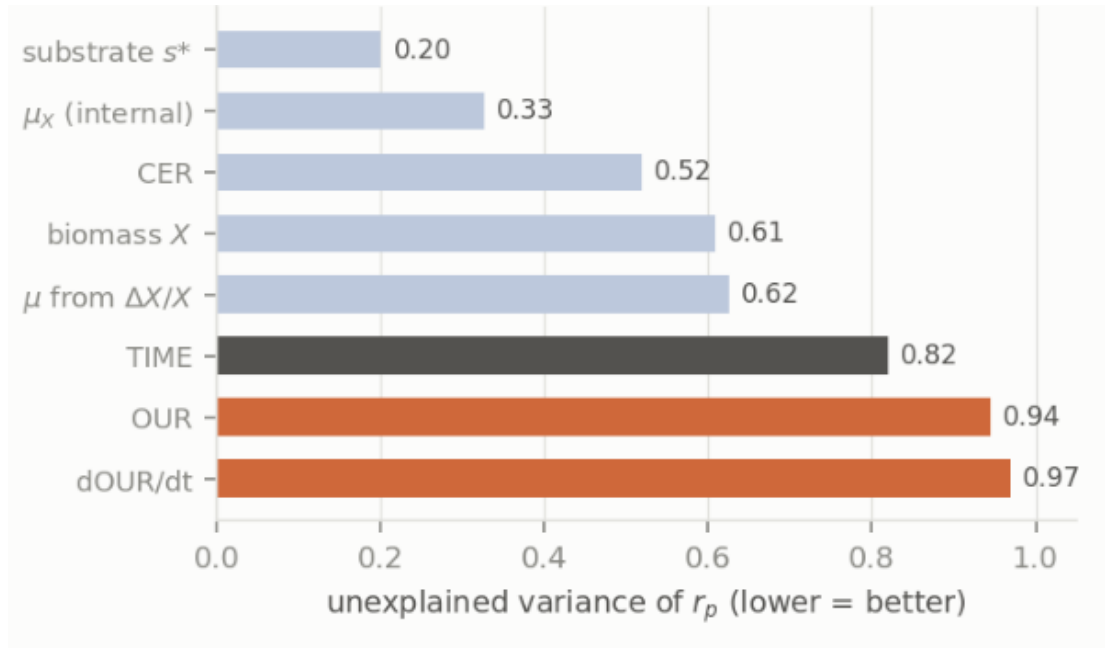


Figure A.7: Unexplained variance test showing which signal best tracks penicillin production across batches.

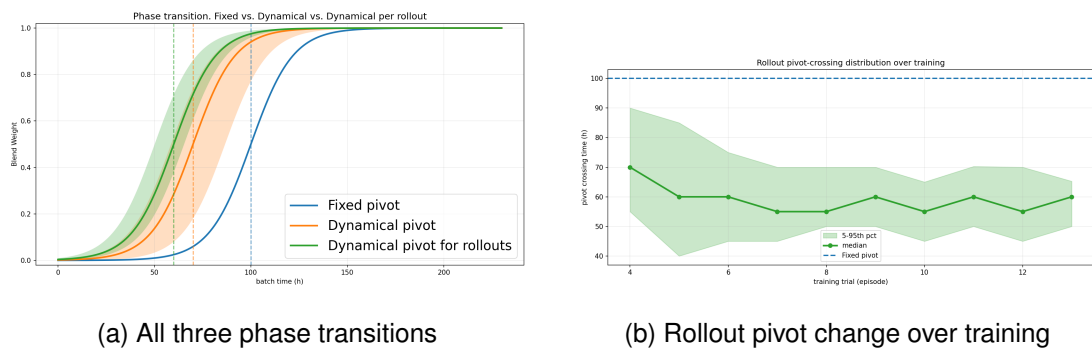


Figure A.9: Phase transitions and rollout pivot transition change over training episodes.

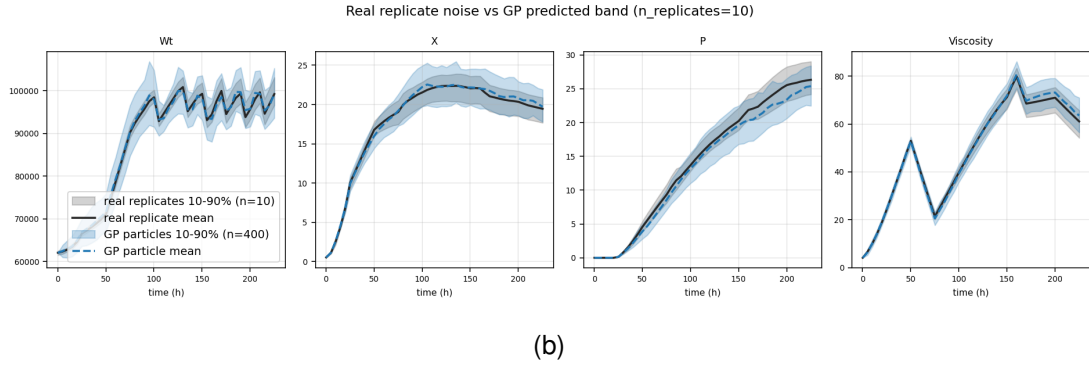
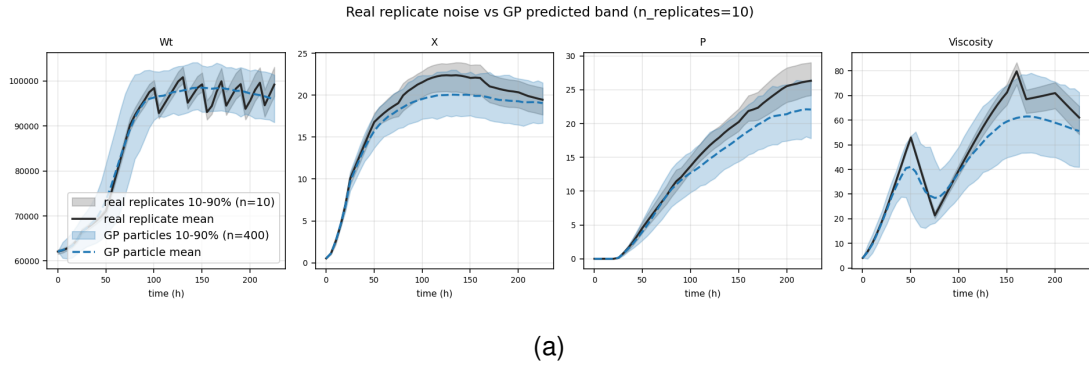


Figure A.10: GP uncertainty intervals on noised-out state trajectories, and the policy prediction. Mass change cost function for single phase GP: (a) **no Time** added, and (b) **Time** added.

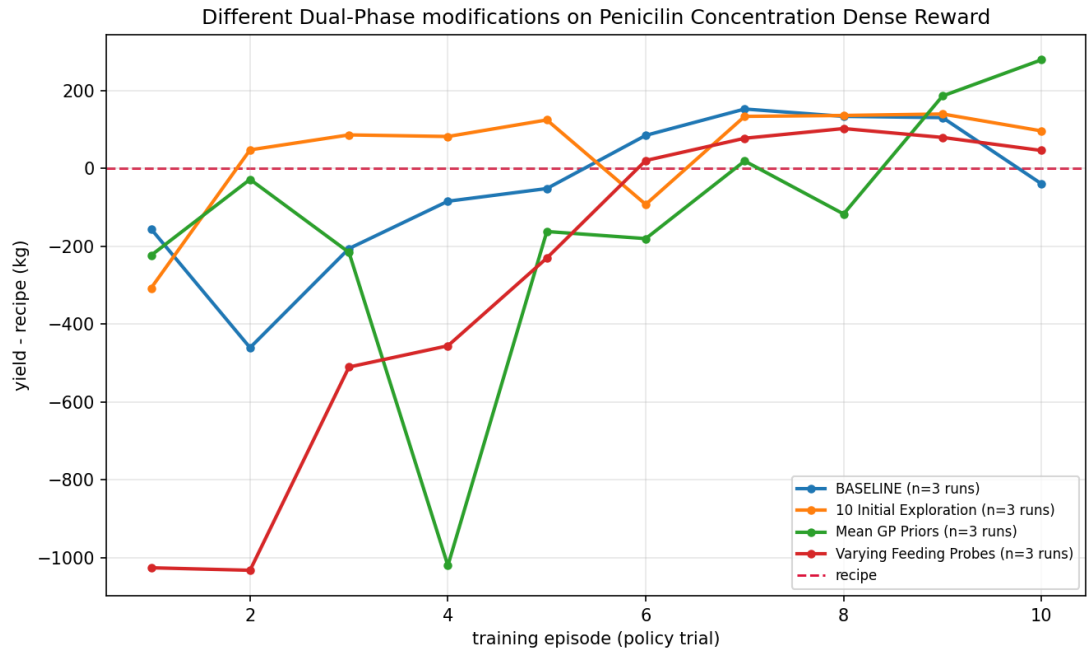


Figure A.11: Multi-phase Additional Experiments on Penicillin concentration dense reward.

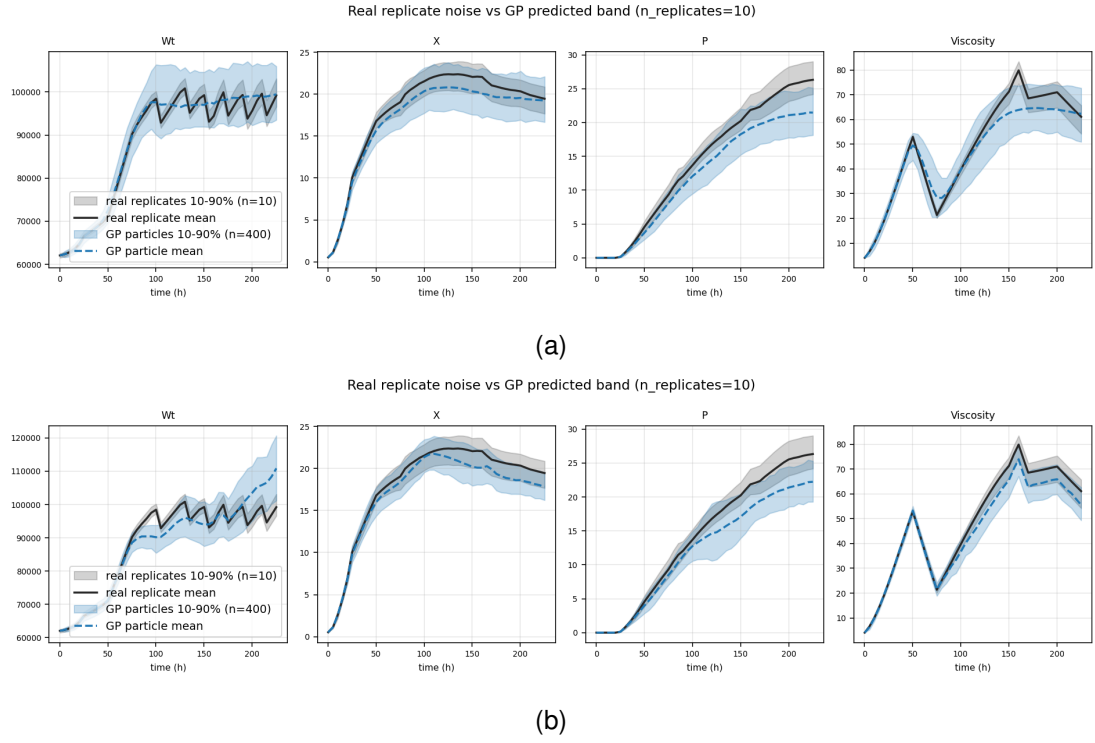


Figure A.12: GP uncertainty intervals on noised-out state trajectories, and the policy prediction. Concentration dense cost function for dual phase GP: (a) **no Time**, and (b) **Time Added**.

Config	model	Mean yield (kg)	SD (batch)	Paired Δ (kg)
No time	Closed-loop	3712.3	320.5	$+3.7 \pm 92.6$
No time	Open-loop	3708.6	308.8	
Added time	Closed-loop	3951.0	356.6	$+33.4 \pm 90.5$
Added time	Open-loop	3917.6	398.6	

Table A.3: Closed-loop versus open-loop (frozen action sequence) execution, 3 training seeds $\times$ 10 held-out batches. Paired Δ is the per-batch closed-open difference (mean $\pm$ SD, $n=30$).

Channel	No time	Added time
Weight	2.58	1.59
Biomass	1.68	1.34
Penicillin	2.23	1.31
Viscosity	3.91	1.26

Table A.4: Ratio of GP predictive band half-width to the real replicate spread (10–90%), averaged over three seeds. A ratio near 1 is well-calibrated. Larger values indicate the model is under-confident. Adding time brings every channel closer to 1.

+45.8% of its peak value and the total achievable upside is only +4.3%. Production sits below it at $z = -0.66$ (80%), where the gradient is 87.5% of peak and +24.3% of upside remains available.

Since the curve is steepest at $|z| = 1$, the production phase operates close to its most substrate-sensitive region. The remaining multipliers in the production rate (DO_2 and PAA inhibition) stayed fixed throughout production, so the substrate is the only varying factor over this window.

λ	mean yield Δ vs recipe	collapsed seeds	peak viscosity	mean P (g/L)	mean Weight (t)	yield/Wt (density)
1.5	-396	2/5 (under-produced)	88 cP	25.5	100.2	30.9
1.0	+193	0	94 cP	27.8	109.4	33.7
0.75	+265	0	95 cP	28.2	110.1	34.2
0.5	+230	0	92 cP	28.2	108.5	34.3

Table A.5: Different constraint strength (λ) tested on 5 different seeds using Penicillin Concentration Dense cost function.

Feed delay Δ (h)	0	20	40	60	80
T1 fixed (100 h)	-39 ± 351	-32 ± 378	-10 ± 329	-2 ± 274	-12 ± 196
T2 (biomass training)	-108 ± 679	-69 ± 734	-58 ± 636	-67 ± 548	-89 ± 414
T3 (biomass rollout)	-96 ± 280	228 ± 143	133 ± 264	60 ± 292	-33 ± 319
Violations, T1	0/30	0/30	0/30	0/30	0/30
Violations, T2	1/30	0/30	0/30	0/30	0/30
Violations, T3	4/30	1/30	0/30	0/30	0/30
Recipe yield (kg)	3563	3361	3141	2908	2620

Table A.6: Yield gain over the recipe (kg, mean $\pm$ SD over three seeds, 30 held-out batches per delay) for the three phase-split methods under the feed-delay disturbance, on the Concentration Dense reward.

Feed delay Δ (h)	0	20	40	60	80
Measured pivot hour	63	75	90	113	132
Recipe yield (kg)	3563	3361	3141	2908	2620

Table A.7: Evaluation if nutritional feed shift Δ displaces the measured growth to production pivot (analytically measured) and progressively reduces the recipe-only yield.

	K_diff (h)	A0 (h)	Proxy (h)
mean	148.66	65.5	121.86
std	4.17	17.5	10.2
min	140.4	49.8	94.6
max	155	91.4	138

Table A.8: K_{diff}, A_0 transition point statistics from 20 different-seed generated batches based on the same recipe.

Predictors	Cross-validated R^2
Intervention only (feed change, timing)	0.696
+ observed state (Wt, X, P, Viscosity)	0.759
+ hidden channels (S, DO2, A0, A1, PAA, CER, μ_P)	0.754

Table A.10: Predicting the measured feed effect on penicillin.

	−50%	−25%	+25%	+50%
Growth ($z = +0.29$)	−9.0%	+3.6%	−18.1%	−43.2%
Production ($z = −0.66$)	−24.3%	−11.8%	+10.2%	+18.0%

Table A.11: Predicted relative change in μ_P (Eq. A.1) at each operating point. Compares sign and ordering with measured Table 4.4.